

Penerapan *Object Detection* Berbasis YOLO untuk Kendali Robot
Vision SCARA pada Proses *Pick and Place*

TUGAS AKHIR

*Diajukan Untuk Memenuhi Salah Satu Persyaratan Guna Memperoleh Gelar
Sarjana Sains Terapan (S.ST)*



Dilham Hidayatul Fajri

NIM. 22130006

**PROGRAM STUDI TEKNIK ELEKTRO INDUSTRI
DEPARTEMEN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

ABSTRAK

Dilham Hidayatul Fajri, 2026. PENERAPAN *OBJECT DETECTION* BERBASIS YOLO UNTUK KENDALI ROBOT *VISION* SCARA PADA PROSES *PICK AND PLACE*

Perkembangan otomasi industri menuntut sistem produksi yang lebih fleksibel, efisien, dan presisi tinggi. Robot SCARA banyak digunakan pada proses *pick and place*, namun masih memiliki keterbatasan dalam menangani objek yang berada pada posisi acak dan lingkungan kerja yang dinamis. Untuk mengatasi permasalahan tersebut, penelitian ini bertujuan menerapkan sistem *vision-guided robotics* (VGR) berbasis YOLO26 yang terintegrasi dengan robot SCARA T3, merancang algoritma transformasi koordinat antara sistem visi dan robot, serta mengevaluasi performa sistem dari aspek akurasi deteksi, komunikasi data, dan efisiensi waktu proses.

Metode penelitian menggunakan pendekatan *Engineering Design Process* (EDP) yang meliputi identifikasi kebutuhan, perancangan, implementasi, pengujian, dan evaluasi sistem. Sistem terdiri atas kamera Logitech C920 sebagai sensor visual, model YOLO26 untuk deteksi objek, Python sebagai pusat pemrosesan data, serta robot Epson SCARA T3 sebagai aktuator. Hasil deteksi objek diproses untuk memperoleh koordinat posisi, kemudian ditransformasikan ke koordinat robot dan dikirim melalui komunikasi TCP/IP untuk menjalankan proses *pick and place* secara otomatis. Selain deteksi *pallet* menggunakan YOLO26, sistem juga menerapkan pengolahan citra berbasis HSV dan deteksi lingkaran untuk identifikasi *workpiece* pada area *pallet*.

Hasil penelitian menunjukkan bahwa sistem VGR berbasis YOLO26 berhasil diimplementasikan dan mampu mengintegrasikan deteksi objek dengan pergerakan robot SCARA T3 secara *real-time*. Sistem dapat mengidentifikasi posisi *workpiece* pada *pallet pick* maupun *pallet place* secara konsisten, dengan tingkat akurasi deteksi mencapai 100% pada seluruh skenario pengujian. Komunikasi TCP/IP yang digunakan memiliki performa responsif dengan nilai *latency* yang rendah, sementara penerapan sistem visi hanya memberikan tambahan waktu yang sangat kecil terhadap *cycle time* proses. Dengan demikian, sistem yang dikembangkan mampu mendukung proses *pick and place* otomatis secara andal, akurat, dan efisien untuk mendukung implementasi otomasi industri berbasis visi komputer.

Kata Kunci: SCARA Robot, Visi Komputer, YOLO, *pick and place*

KATA PENGANTAR



Puji dan syukur penulis panjatkan atas kehadiran Allah SWT yang telah memberikan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Tugas Akhir ini dengan “Penerapan *Object Detection* Berbasis YOLO untuk Kendali Robot *Vision SCARA* pada Proses *Pick and Place*”. Tugas akhir ini dibuat sebagai salah satu syarat untuk menyelesaikan Program Studi D IV Teknik Elektro Industri Departemen Teknik Elektro Fakultas Teknik Universitas Negeri Padang.

Pada penelitian dan penulisan Tugas Akhir ini, penulis banyak mendapat bantuan dan bimbingan dari berbagai pihak. Pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih kepada:

1. Kedua Orang Tua dan Saudara yang telah memberikan dorongan dan semangat, bantuan materi, do'a dalam menyelesaikan tugas akhir ini.
2. Bapak Krismadinata, S.T., M.T., Ph.D selaku Rektor Universitas Negeri Padang, beserta wakil rektor semua bidang.
3. Bapak Dr. Muhammad Anwar, S.Pd, MT. selaku Dekan Fakultas Teknik Universitas Negeri Padang.
4. Bapak Dr. Mukhlidi Muskhir, S.Pd., M.Kom. selaku Kepala Departemen Teknik Elektro Universitas Negeri Padang
5. Bapak Dr. Muldi Yuhendri, S.Pd, M.T selaku Kepala Program Studi Teknik Elektro Industri Universitas Negeri Padang.
6. Bapak Ir. Risfendra, S.Pd, MT, Ph.D selaku Dosen Pembimbing yang telah memberikan arahan, motivasi serta semangat dalam mengerjakan Tugas Akhir ini.
7. Bapak Dr. Muldiyuhendri, S.Pd., MT dan bapak Fadhli Ranuharja, S.Pd., M.Pd.T. selaku dosen pengarah yang telah membantu dalam memberikan arahan, motivasi dan semangat dalam memberikan arah penelitian.

8. Bapak dan Ibu Dosen serta seluruh staf Jurusan Teknik Elektro Fakultas Teknik Universitas Negeri Padang.
9. Bapak Eko Purnama, S.T, MM selaku Direktur PT. UNPMES beserta staf yang telah memberikan kesempatan untuk menimba ilmu di PT. UNPMES sehingga penulis mendapatkan kesempatan menyelesaikan Tugas Akhir dari *project* PT. UNPMES
10. Teman-teman seperjuangan dan semua pihak yang telah membantu penulis untuk mewujudkan Tugas Akhir ini dan menyelesaikan studi.

Penulis menyadari bahwa penyusunan Tugas Akhir ini masih jauh dari kata sempurna. Oleh karena itu segala saran dan kritik yang bersifat membangun sangat penulis harapkan. Semoga tugas akhir ini dapat bermanfaat dan menjadi tambahan ilmu pengetahuan bagi yang membacanya.

Padang, 17 Oktober 2025

Dilham Hidayatul Fajri

22130006

DAFTAR ISI

	Hal
ABSTRAK	i
KATA PENGANTAR.....	ii
DAFTAR ISI.....	iv
DAFTAR GAMBAR	vi
DAFTAR TABEL	vi
BAB I PENDAHULUAN.....	1
A. Latar Belakang	1
B. Identifikasi Masalah	4
C. Rumusan Masalah	4
D. Batasan Masalah.....	5
E. Tujuan	5
F. Manfaat.....	5
BAB II LANDASAN TEORI	7
A. Otomasi Industri.....	7
B. Robot Industri	8
C. Robot Epson SCARA T3	10
D. Kamera <i>WebCam</i>	13
E. <i>Computer Vision</i>	13
F. YOLO (<i>You Look Only Once</i>)	14
G. <i>Circle detection</i>	16
H. <i>Vision Guide Robotics</i>	20
I. <i>TCP/IP Communication</i>	21
J. <i>Sistem Pneumatik</i>	24
K. <i>Power Supply Unit</i>	26
L. Python	27
M. <i>General User Interface</i>	27
N. Penelitian Terdahulu	27
BAB III METODE PERANCANGAN.....	30
A. Identifikasi Kebutuhan	30
B. Perancangan Sistem.....	33

C. Implementasi Sistem	50
D. Pengujian.....	51
E. Evaluasi.....	53
F. <i>Time Schedule</i>	54
G. Rancangan Anggaran Biaya	55
BAB IV HASIL DAN PEMBAHASAN.....	56
A. Analisis kinerja Model YOLOv26	56
B. Pengujian dan analisis <i>latency</i> komunikasi Data TCP/IP	64
C. Pengujian dan analisis pengaruh <i>Cycle time</i>	69
D. Pengujian dan hasil deteksi <i>pallet</i> dan <i>workpiece</i>	71
BAB V KESIMPULAN DAN SARAN	77
A. Kesimpulan.....	77
B. Saran	78
DAFTAR PUSTAKA.....	79
LAMPIRAN.....	83
Lampiran 1. Program Epson RC+	83
Lampiran 2. <i>Repository</i> Github.....	83
Lampiran 3. Buku Panduan.....	83

DAFTAR GAMBAR

Gambar	Hal
Gambar 2. 1. Epson Scara T3-B401S Robot.....	11
Gambar 2. 2. Struktur <i>arm</i> dan <i>joint</i> robot Scara T3-B401S	11
Gambar 2. 3. Kamera <i>Logitech</i> C920e.....	13
Gambar 2. 4. <i>Bounding box</i> dengan YOLO	14
Gambar 2. 5. Arsitektur YOLOv26	16
Gambar 2. 6. Deteksi lingkaran.....	17
Gambar 2. 7. Model HSV	18
Gambar 2. 8. <i>Morphological Convolutional Neural Network Process</i>	19
Gambar 2. 9. Kabel Ethernet.....	23
Gambar 2. 10. Konverter Ethernet to Type C	24
Gambar 2. 11. Bentuk dan simbol <i>Solenoid valve</i>	24
Gambar 2. 12. Bentuk dan simbol <i>Air Filter Regulator</i>	25
Gambar 2. 13. Bentuk dan simbol <i>Vacuum Generator</i>	25
Gambar 2. 14. Bentuk dan simbol <i>Vacuum Suction Cup</i>	26
Gambar 2. 15. Omron <i>Power Supply Unit</i>	26
Gambar 3. 1. Metode <i>Engineering Design Process</i>	30
Gambar 3. 2. Blok diagram.....	33
Gambar 3. 3. Alur kerja sistem.....	36
Gambar 3. 4. Tampilan area	36
Gambar 3. 5. <i>Flowchart</i> program python.....	37
Gambar 3. 6. <i>Flowchart</i> program RC+	38
Gambar 3. 7. Video <i>dataset</i>	40
Gambar 3. 8. Hasil gambar dari <i>frame</i> video	40
Gambar 3. 9. Gambar di <i>upload</i> ke <i>roboflow</i>	41
Gambar 3. 10. Proses <i>labelling</i> gambar	41
Gambar 3. 11. Partisi <i>dataset</i>	42
Gambar 3. 12. Training data menggunakan <i>Google Collab</i>	43
Gambar 3. 13. Tampilan GUI.....	46
Gambar 3. 14. Tampilan <i>Jog and Teach</i>	47
Gambar 3. 15. Tampilan I/O <i>Monitor</i>	48
Gambar 3. 16. Desain Sistem: a. Nama <i>part</i> . b. Dimensi	49
Gambar 3. 17. Rangkaian <i>Power</i>	49
Gambar 3. 18. Rangkaian <i>Vacuum Suction Cup</i>	50
Gambar 4. 1. Grafik <i>loss</i> (a) <i>box_loss</i> ; (b) <i>cls_loss</i> ; (c) <i>dfl_loss</i>	58
Gambar 4. 2. Kurva Presisi terhadap <i>confidence</i>	60
Gambar 4. 3. Kurva <i>Recall</i> terhadap <i>confidence</i>	61
Gambar 4. 4. Kurva F1 terhadap <i>confidence</i>	62
Gambar 4. 5. <i>Confusion Matrix</i>	63
Gambar 4. 6. Kondisi <i>workpiece</i> saat pengujian <i>cycle time</i>	70

DAFTAR TABEL

Tabel	Hal
Tabel 2. 1. Spesifikasi Epson Robot Scara T3	12
Tabel 2. 2. Standar Komunikasi TCP/IP Industri	22
Tabel 3. 1. Tampilan <i>Grid</i>	44
Tabel 3. 2. <i>Time Schedule</i>	54
Tabel 3. 3. Rancangan Anggaran Biaya	55
Tabel 4. 1. Hasil konvergensi loss model.....	57
Tabel 4. 2. Data <i>Latency</i> percobaan ke-1	64
Tabel 4. 3. Data <i>Latency</i> percobaan ke-2	65
Tabel 4. 4. Data <i>Latency</i> percobaan ke-3	66
Tabel 4. 5. Data <i>Latency</i> percobaan ke-4	66
Tabel 4. 6. Data <i>Latency</i> percobaan ke-5	67
Tabel 4. 7. Rekapitulasi latency semua percobaan.....	67
Tabel 4. 8. Pola periodik <i>latency</i>	68
Tabel 4. 9. Hasil pengujian <i>cycle time</i> dengan <i>vision</i>	70
Tabel 4. 10. Hasil pengujian <i>cycle time</i> tanpa <i>vision</i>	70
Tabel 4. 11. Hasil pengujian deteksi <i>pallet pick</i>	71
Tabel 4. 12. Hasil pengujian deteksi <i>pallet place</i>	73

BAB I

PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi otomasi industri semakin pesat seiring dengan kebutuhan akan efisiensi, fleksibilitas, dan ketepatan produksi dalam era revolusi industri keempat. Implementasi teknologi otomasi dapat meningkatkan produktivitas manufaktur hingga 20-35% sambil mengurangi biaya operasional secara signifikan (Djarmika, 2023). Industri manufaktur modern, khususnya pada sektor elektronik, otomotif, dan pengemasan, proses *pick and place* masih menjadi titik kritis dalam rantai produksi. Banyak perusahaan menghadapi permasalahan berupa waktu siklus yang tidak konsisten, ketergantungan pada operator manusia, serta tingkat kesalahan posisi objek yang tinggi akibat kelelahan dan variasi keterampilan tenaga kerja. Kondisi ini semakin kompleks ketika volume produksi meningkat dan toleransi kesalahan semakin kecil.

Perkembangan teknologi otomasi industri semakin pesat sejalan dengan kebutuhan akan efisiensi, fleksibilitas, dan ketepatan produksi. Salah satu aspek penting dalam otomasi adalah robot industri, khususnya robot SCARA (*Selective Compliance Assembly Robot Arm*) yang banyak digunakan dalam proses *pick and place*, perakitan elektronik, hingga *packaging*. Robot SCARA memiliki keunggulan berupa kecepatan tinggi, presisi posisi yang baik, dan kestabilan dalam lintasan horizontal. Fenomena tersebut mendorong industri beralih ke robot SCARA, yang secara spesifik dirancang untuk pekerjaan berulang dengan lintasan horizontal, kecepatan tinggi, dan presisi posisi.

International Federation of Robotics (IFR) melaporkan bahwa penggunaan robot SCARA mengalami pertumbuhan 15% per tahun selama periode 2019-2023, dengan dominasi penggunaan di industri elektronik (45%), otomotif (25%), dan *food processing* (20%). Robot SCARA memiliki keunggulan berupa kecepatan tinggi (*cycle time* < 0.5 detik), presisi posisi yang baik ($\pm 0.01\text{mm}$), dan kestabilan dalam lintasan horizontal, membuatnya ideal untuk aplikasi yang memerlukan *throughput* tinggi dengan akurasi konsisten (Kapusi et al., 2022).

Untuk mengatasi keterbatasan tersebut dan menjadikan robot SCARA lebih adaptif, sistem *vision-guided robotics* (VGR) telah dikembangkan sebagai solusi inovatif. VGR memungkinkan robot menerima *input* berupa koordinat objek dari visi komputer secara *real-time*, yang sangat penting untuk penanganan objek yang tidak beraturan, posisi acak. Implementasi VGR dapat meningkatkan fleksibilitas sistem robotik hingga 70% dibandingkan sistem konvensional (Singh et al., 2022). VGR memungkinkan robot menerima input berupa koordinat objek dari visi komputer secara *real-time*, yang sangat penting untuk penanganan objek yang tidak beraturan ataupun bergerak.

Permasalahan nyata yang dihadapi industri modern adalah tingginya tingkat variasi produk serta meningkatnya kebutuhan akan *mass customization* (Kim & Lee, 2023). Misalnya, pada sektor perakitan elektronik dan otomotif, variasi bentuk komponen dan orientasi objek di *conveyor* sering kali menghambat efektivitas robot konvensional. Sistem robot tradisional yang hanya mengandalkan koordinat terprogram sebelumnya sering gagal menghadapi kondisi produksi yang dinamis, seperti objek dengan posisi acak, perubahan pencahayaan, maupun variasi geometri produk. Hal ini menciptakan *bottleneck* yang menurunkan *throughput* produksi sekaligus meningkatkan biaya operasional.

Fenomena serupa juga ditemukan pada lingkungan *Teaching Factory* di perguruan tinggi vokasi, termasuk di Universitas Negeri Padang. Meskipun telah tersedia robot industri SCARA sebagai sarana pembelajaran, pemanfaatannya masih terbatas pada pemrograman gerak konvensional berbasis teaching point. Proses ini belum merepresentasikan kondisi industri sesungguhnya yang menuntut sistem robot bekerja secara adaptif terhadap variasi posisi dan bentuk objek.

Akibatnya, mahasiswa cenderung memahami robot sebagai sistem yang kaku dan statis, bukan sebagai bagian dari ekosistem otomasi cerdas. Keterbatasan modul pembelajaran berbasis *vision-guided robotics* menyebabkan lulusan belum sepenuhnya siap menghadapi kebutuhan industri yang telah mengadopsi sistem berbasis visi dan kecerdasan buatan. Oleh karena

itu, diperlukan pengembangan sistem pembelajaran yang mampu menjembatani kebutuhan industri dan kompetensi lulusan melalui implementasi VGR yang aplikatif. Namun, untuk mencapai tingkat otomasi yang lebih cerdas atau VGR, robot industri memerlukan kemampuan persepsi visual. Hal ini mendorong integrasi *computer vision* berbasis *deep learning*, seperti metode *You Only Look Once* (YOLO), dalam sistem kendali robot. YOLO dikenal karena kemampuannya melakukan deteksi objek secara *real-time* dengan akurasi tinggi (Ku et al., 2022; Sapkota et al., 2025).

Pemilihan metode YOLO (*You Only Look Once*) dalam penelitian ini didasarkan pada performanya yang konsisten unggul dalam mendeteksi objek secara *real-time* (Gheorghe et al., 2024; Guan, 2023). Dibandingkan dengan algoritma lain seperti Faster R-CNN (*Region-based Convolutional Neural Network*) atau SSD (*Single Shot Detector*), YOLO menawarkan *trade-off* terbaik antara akurasi dan kecepatan inferensi, sehingga ideal untuk aplikasi industri yang membutuhkan deteksi cepat dan presisi (Olorunshola et al., 2023; Sarma et al., 2024; Shende & Khandelwal, 2025). Evolusi YOLO dari versi awal hingga terbaru (YOLOv4, YOLOv5, hingga YOLOv6) semakin memperkuat kapabilitasnya dalam mendukung sistem VGR berbasis *deep learning* (Kang et al., 2025). Hal ini sejalan dengan penelitian terbaru yang menegaskan YOLO sebagai algoritma yang paling efisien untuk implementasi robotik industri.

Berdasarkan permasalahan yang telah diidentifikasi, penelitian ini merancang sebuah sistem *vision-guided robot* SCARA berbasis YOLO untuk aplikasi *pick and place* industri yang mampu menjawab tantangan integrasi *real-time* sekaligus memberikan solusi praktis bagi implementasi smart *manufacturing*. Penelitian ini menggunakan *framework* terintegrasi yang menggabungkan kemampuan deteksi objek YOLO dengan presisi dan kecepatan robot SCARA, dengan fokus pada optimisasi arsitektur komunikasi TCP/IP untuk mencapai latensi minimal serta pengembangan algoritma koordinasi yang mampu menyelaraskan hasil deteksi YOLO dengan sistem koordinat robot secara *real-time*.

Kontribusi utama penelitian ini diharapkan mencakup penerapan model integrasi yang dapat di replikasi untuk implementasi VGR di berbagai *setting* industri, penyusunan *benchmark* performa sistem VGR berbasis YOLO khususnya pada aplikasi robot SCARA, serta penyediaan solusi praktis yang layak diimplementasikan industri dengan investasi yang rasional. Selain itu, penelitian ini diharapkan dapat menjadi *knowledge base* bagi pengembangan sistem otomasi cerdas di Indonesia. Dengan demikian, hasil penelitian ini tidak hanya memberikan sumbangan pada pengembangan teoretis di bidang teknologi otomasi, tetapi juga menghadirkan *impact* praktis yang mendukung transformasi digital industri manufaktur Indonesia menuju era Industri 4.0 yang lebih kompetitif dan berkelanjutan.

B. Identifikasi Masalah

Perkembangan teknologi otomasi industri menuntut sistem produksi yang lebih fleksibel, efisien, dan presisi tinggi. Robot SCARA T3 banyak digunakan karena kecepatan dan akurasinya, namun masih memiliki keterbatasan dalam menangani objek yang tidak terstruktur. Keterbatasan robot SCARA T3 konvensional terletak pada kemampuannya yang hanya dapat beroperasi dalam lingkungan terstruktur dengan objek yang telah diprogram sebelumnya. Hal ini menciptakan gap antara fleksibilitas yang dibutuhkan industri modern dengan rigiditas sistem robotik tradisional. Hal ini menciptakan kebutuhan akan integrasi teknologi persepsi visual, seperti *vision-guided robotics* (VGR), yang didukung oleh *computer vision* berbasis *deep learning* seperti YOLO. Namun, implementasi *real-time* sistem VGR berbasis YOLO pada robot SCARA T3 masih menghadapi tantangan dari sisi integrasi koordinat, dan sinkronisasi deteksi dengan gerakan robot.

C. Rumusan Masalah

Berdasarkan latar belakang dan tujuan penelitian, maka rumusan masalah yang diajukan adalah:

1. Bagaimana menerapkan sistem *vision-guided robotics* (VGR) berbasis YOLO yang dapat diintegrasikan dengan robot SCARA T3 untuk aplikasi *pick and place*?

2. Bagaimana merancang dan mengimplementasikan algoritma transformasi koordinat serta sinkronisasi agar hasil deteksi objek dapat diterjemahkan ke dalam pergerakan robot SCARA T3 dengan akurat?
3. Bagaimana mengukur performa sistem VGR berbasis YOLO dalam hal efisiensi waktu, akurasi deteksi, dan kecepatan penanganan objek?

D. Batasan Masalah

Agar penelitian lebih terfokus dan sesuai dengan ruang lingkup yang ditetapkan, maka batasan masalah dalam penelitian ini adalah:

1. Sistem *vision* hanya menggunakan metode deteksi objek berbasis YOLOv26, tanpa membandingkan dengan metode lain.
2. Robot yang digunakan terbatas pada jenis SCARA Epson T3.
3. Objek yang diuji pada sistem *pick and place* berupa benda dengan bentuk sederhana *workpiece* lingkaran yang diletakkan pada *pallet*.
4. Penilaian performa sistem hanya difokuskan pada parameter: evaluasi model YOLOv26 berupa hasil *training*, presisi, *recall*, *F1 Score*, *Confusion matrix*, *latency*, dan *cycle time*.
5. Penelitian dilakukan dalam lingkungan *Teaching Factory*, bukan pada kondisi industri sesungguhnya.

E. Tujuan

Adapun tujuan dalam penelitian ini adalah sebagai berikut

1. Menerapkan sistem *vision guided robotics* (VGR) berbasis YOLO yang terintegrasi dengan robot SCARA T3 untuk aplikasi *pick and place*.
2. Membuat algoritma transformasi koordinat dan sinkronisasi yang mampu menghubungkan hasil deteksi objek dengan pergerakan robot SCARA secara akurat.
3. Mengukur performa sistem VGR berbasis YOLO dalam konteks efisiensi waktu, akurasi deteksi, dan kecepatan penanganan objek.

F. Manfaat

Manfaat yang diharapkan dari penelitian ini tidak hanya bersifat akademis, tetapi juga memberikan dampak praktis dan aplikatif, baik bagi industri maupun lingkungan *Teaching Factory*, sebagai berikut:

1. Manfaat bagi Industri.
 - a. Memberikan solusi penerapan dalam meningkatkan fleksibilitas pada proses *pick and place* untuk robot SCARA T3 melalui implementasi teknologi *vision guided robotic* berbasis YOLO, sehingga mampu menangani objek dengan posisi yang bervariasi tanpa pemograman ulang.
 - b. Mengurangi ketergantungan dengan sistem teaching point yang membutuhkan proses teaching dan pemograman yang rumit.
 - c. Mendukung penerapan produksi fleksibel (*flexible manufacturing system*), dimana dapat menangani beragam jenis variasi produk.
 - d. Sebagai salah satu solusi penerapan vision dengan biaya kecil dengan kamera konvensional tanpa menggunakan modul resmi.
2. Manfaat bagi *Teaching Factory*
 - a. Sebagai media pembelajaran *prototype* untuk penerapan teknologi *vision guide robotic* bagi mahasiswa *Teaching Factory*.
3. Manfaat bagi pengembangan Teknologi dan Penelitian
 - a. Memberikan kontribusi ilmiah dalam pengembangan penelitian di bidang dan robotik industri.
 - b. Menjadi referensi akademik terkait integrasi sistem deteksi objek dengan pergerakan robot secara *real time*.

BAB II

LANDASAN TEORI

A. Otomasi Industri

Otomatisasi industri merupakan suatu pendekatan yang memanfaatkan teknologi untuk menggantikan atau mengurangi peran tenaga kerja manual dalam proses produksi (Pramudita et al., 2024). Tujuan utama dari penerapan otomatisasi ini adalah untuk meningkatkan efisiensi, produktivitas, serta kualitas produk secara keseluruhan, yang pada akhirnya berkontribusi pada peningkatan daya saing perusahaan di pasar global. Dalam konteks industri manufaktur, otomatisasi mengacu pada penggunaan berbagai perangkat teknologi canggih seperti robot, sistem kontrol otomatis, dan perangkat lunak berbasis AI untuk mengelola dan mengontrol proses produksi secara lebih akurat dan efisien. Salah satu keuntungan utama dari otomatisasi adalah kemampuannya untuk mengurangi ketergantungan terhadap tenaga kerja manusia yang rentan terhadap kesalahan. Dengan demikian, otomatisasi dapat mengurangi tingkat kesalahan produksi, mengoptimalkan penggunaan energi, serta mempercepat waktu siklus produksi.

Menurut (Djarmika, 2023), otomatisasi industri memiliki potensi untuk meningkatkan produktivitas manufaktur sebesar 20–35%. Hal ini dapat dicapai melalui pengurangan kesalahan manusia yang umumnya terjadi dalam proses produksi manual. Selain itu, dengan otomatisasi, penggunaan energi dapat lebih dioptimalkan, sehingga perusahaan dapat mengurangi pemborosan energi dan meningkatkan efisiensi operasional. Keuntungan lainnya adalah waktu siklus produksi yang semakin pendek, yang memungkinkan perusahaan untuk menghasilkan produk dalam jumlah lebih banyak dan lebih cepat, meningkatkan kapasitas produksi secara keseluruhan. Secara keseluruhan, penerapan otomatisasi di industri tidak hanya mengarah pada pengurangan biaya operasional, tetapi juga memberikan dampak positif terhadap peningkatan kualitas produk yang lebih konsisten dan terstandarisasi.

B. Robot Industri

Robot industri adalah perangkat otomatisasi yang dirancang dan digunakan untuk menyelesaikan tugas-tugas berulang secara otonom dengan tingkat presisi, akurasi, dan kecepatan yang tinggi dalam lingkungan manufaktur maupun non-manufaktur. Kehadiran robot industri dalam ekosistem produksi modern bukan lagi sekadar pelengkap, melainkan telah menjadi komponen inti yang menentukan daya saing industri di era Revolusi Industri 4.0. Robot industri mampu bekerja secara terus-menerus tanpa kelelahan, meminimalkan kesalahan manusia, dan menghasilkan *output* yang konsisten dalam jumlah besar dengan toleransi yang sangat ketat.

Secara teknis, terdapat beberapa parameter utama yang mendefinisikan kinerja robot industri. Presisi mengacu pada kemampuan robot untuk mencapai posisi target dengan penyimpangan minimal dari titik yang ditentukan, sementara akurasi menunjukkan kesesuaian antara posisi aktual yang dicapai robot dengan titik referensi yang diinginkan (Izzulhaq & Budiyanto, 2025). Kedua parameter ini sering kali dibedakan secara konseptual: robot dapat memiliki presisi tinggi namun akurasi rendah apabila ia secara konsisten mencapai posisi yang sama tetapi posisi tersebut menyimpang dari target yang sebenarnya, atau sebaliknya.

Repeatability, di sisi lain, menggambarkan konsistensi robot dalam mengulangi gerakan yang sama secara berulang kali dengan deviasi yang sangat kecil. Parameter ini merupakan aspek yang paling krusial dalam aplikasi produksi massal, karena tingkat *repeatability* yang tinggi menjamin bahwa setiap unit produk yang dihasilkan memiliki kualitas yang seragam. Menurut Siciliano et al. (2009) dalam *Robotics: Modelling, Planning and Control*, *repeatability* merupakan salah satu indikator terpenting dalam pemilihan robot industri untuk aplikasi perakitan presisi, karena nilai *repeatability* yang rendah secara langsung berkontribusi pada peningkatan *yield rate* produksi dan pengurangan produk cacat (*defect rate*).

Selain ketiga parameter di atas, kinerja robot industri juga ditentukan oleh sejumlah karakteristik teknis lainnya yang tidak kalah penting. Ketahanan

(*durability*) mencerminkan kemampuan robot untuk bertahan dalam penggunaan jangka panjang di lingkungan kerja yang keras, termasuk paparan terhadap debu, kelembapan, getaran mekanis, dan perubahan suhu yang signifikan. Keandalan (*reliability*) berkaitan dengan kemampuan sistem untuk mempertahankan stabilitas operasional tanpa kegagalan yang sering, yang secara langsung mempengaruhi *uptime* lini produksi. Dalam industri manufaktur modern, kegagalan robot yang tidak terencana dapat mengakibatkan kerugian produksi yang sangat besar, sehingga keandalan menjadi pertimbangan utama dalam pemilihan sistem robotik.

Kapasitas beban (*payload capacity*) menentukan stabilitas kinerja robot saat menangani objek dengan berat tertentu. Setiap robot dirancang untuk beroperasi optimal dalam rentang beban tertentu, dan pengoperasian di luar rentang ini dapat menyebabkan penurunan akurasi, keausan komponen yang dipercepat, bahkan kegagalan sistem. Craig (2005) dalam *Introduction to Robotics: Mechanics and Control* menjelaskan bahwa kapasitas beban robot secara fundamental dibatasi oleh spesifikasi mekanis *joint* dan *actuator*-nya, serta berkaitan erat dengan kecepatan operasional karena beban inersia yang lebih tinggi memerlukan gaya dan torsi yang lebih besar untuk mencapai akselerasi dan deselerasi yang sama. Oleh karena itu, pemilihan robot dengan kapasitas beban yang sesuai dengan aplikasi merupakan syarat mutlak untuk memastikan kinerja yang optimal dan umur pakai sistem yang panjang.

Kecepatan siklus (*cycle time*) merupakan parameter yang secara langsung mempengaruhi efisiensi dan produktivitas lini produksi. Kecepatan siklus yang lebih pendek berarti robot mampu menyelesaikan lebih banyak tugas dalam satuan waktu yang sama, sehingga meningkatkan *throughput* produksi secara keseluruhan. Namun demikian, peningkatan kecepatan siklus harus selalu diimbangi dengan pertimbangan terhadap akurasi dan keandalan, karena operasi pada kecepatan sangat tinggi dapat meningkatkan risiko kesalahan posisi dan mempercepat keausan mekanis.

Berdasarkan konfigurasi mekanisnya, robot industri dapat diklasifikasikan ke dalam beberapa jenis utama, yaitu robot *Cartesian*, robot silindris, robot

spherical, robot SCARA (*Selective Compliance Assembly Robot Arm*), robot *articulated* (lengan berengsel), dan robot *delta* (paralel). Masing-masing konfigurasi memiliki keunggulan dan keterbatasan tersendiri yang menentukan kesesuaiannya dengan jenis aplikasi tertentu. Robot SCARA, misalnya, unggul dalam aplikasi *pick and place* dan perakitan horizontal karena memiliki kekakuan tinggi pada arah vertikal namun fleksibel pada bidang horizontal, menjadikannya sangat efisien untuk tugas-tugas yang melibatkan pemindahan objek pada ketinggian yang sama dengan kecepatan tinggi dan presisi baik.

Dengan kombinasi seluruh karakteristik tersebut, robot industri memainkan peran yang semakin krusial dalam mendukung otomatisasi manufaktur modern. Di era Industri 4.0, robot industri tidak lagi berdiri sendiri sebagai perangkat mekanis yang terisolasi, melainkan telah berkembang menjadi bagian dari ekosistem otomasi terintegrasi yang terhubung dengan sistem *cyber-physical*, kecerdasan buatan, *Internet of Things* (IoT), dan *big data analytics*. Integrasi ini memungkinkan robot industri untuk tidak hanya melakukan tugas-tugas fisik yang berulang, tetapi juga berpartisipasi dalam pengambilan keputusan berbasis data secara *real-time*, beradaptasi terhadap perubahan permintaan produksi, dan berkolaborasi dengan operator manusia dalam lingkungan kerja yang dinamis dan fleksibel.

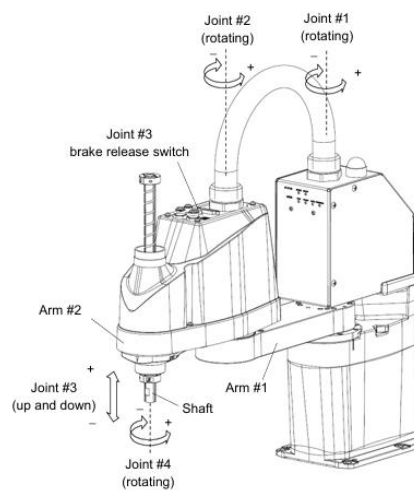
C. Robot Epson SCARA T3

Robot Epson SCARA T3 adalah salah satu robot industri kelas *Selective Compliance Assembly Robot Arm* yang dirancang khusus untuk aplikasi yang membutuhkan kecepatan dan presisi tinggi, seperti *pick and place*, perakitan, dan inspeksi. SCARA T3 sangat cocok untuk tugas manipulasi di industri elektronik dan otomotif karena keunggulannya dalam gerakan horizontal yang stabil. Pada Gambar 2.1 ditunjukkan bentuk robot Epson SCARA T3 yang termasuk dalam kategori robot dengan konfigurasi *Selective Compliance Assembly Robot Arm*.



Gambar 2. 1. Epson Scara T3-B401S Robot
(Sumber: epson.com)

Secara teknis, Epson SCARA T3 dilengkapi dengan kemampuan presisi posisi mencapai $\pm 0,01$ mm serta *repeatability* tinggi, menjadikannya andal untuk aplikasi yang menuntut konsistensi. Robot ini memiliki kapasitas beban (*payload*) hingga 3 kg dengan jangkauan lengan sekitar 400 mm, cukup untuk menangani komponen berukuran kecil hingga menengah. Selain itu, salah satu keunggulan SCARA T3 adalah penggunaan *integrated control system* yang tidak memerlukan *controller* eksternal, sehingga instalasi lebih sederhana dan biaya investasi lebih efisien. Gambar 2.2 menunjukkan arah gerakan setiap *joint* dengan gerakan panah, dengan *joint* pertama ditunjukkan di lengan pertama dan *joint* keempat ditunjukkan di lengan paling akhir.



Gambar 2. 2. Struktur *arm* dan *joint* robot Scara T3-B401S
(Sumber: epson.com)

1. Spesifikasi Robot Epson Scara T3

Spesifikasi robot ditampilkan ada Tabel 2.1, yang mencakup jangkauan maksimum, kecepatan operasi, kapasitas beban, tingkat *repeatability*, serta jenis *controller* terintegrasi. Informasi ini diperlukan untuk mengetahui kemampuan batas kerja robot, sebab apabila digunakan melebihi spesifikasi yang telah ditentukan maka sistem robot dapat mengalami gangguan atau *error*. Apabila hal tersebut terjadi, robot perlu dilakukan proses reset agar kembali dapat beroperasi dengan normal. Dengan memahami spesifikasi tersebut, operator dapat memastikan pemakaian robot sesuai kapasitasnya sehingga kinerja tetap optimal dan umur pakai sistem dapat terjaga.

Tabel 2. 1. Spesifikasi Epson Robot Scara T3

<i>Mounting type</i>	<i>Table top</i>	
<i>Arm length</i>	<i>Arm #1</i>	400 mm
<i>Weight (Cables not included)</i>	16 kg	
<i>Repeatability</i>	<i>Joints #1, #2</i>	± 0.020 mm
	<i>Joint #3</i>	± 0.020 mm
	<i>Joint #4</i>	± 0.020 deg
<i>Max. Motion Range</i>	<i>Joint #1</i>	± 132 deg
	<i>Joint #2</i>	± 141 deg
	<i>Joint #3</i>	150 mm
	<i>Joint #4</i>	± 360 deg
<i>Payload</i>	<i>Rated</i>	1 kg
	<i>Maximum</i>	3 kg
<i>Standard cycle time</i>		0.52 sec
<i>Joint #4 allowable</i>	<i>Rated</i>	0.003 kgm ²
<i>Moment of inertia</i>	<i>Maximum</i>	0.010 kgm ²
<i>Joint #3 downward force</i>		83 N
<i>Electric Lines</i>		<i>Hand I/O: IN6/OUT4 (D-sub 15-Pin) / User I/O: IN18/OUT12</i>
<i>Pneumatic lines</i>		$\varnothing 6$ mm $\times$ 2, $\varnothing 4$ mm $\times$ 1
<i>Installation environment</i>		<i>Standard</i>
<i>Available controllers</i>		<i>Built-in</i>
<i>Safety standards</i>		CE Mark: EMC Directive, Machinery Directive, RoHS Directive ANSI/RIA R15.06-2012 NFPA 79 (2007 Edition)

(Sumber: epson.com)

D. Kamera *WebCam*

Kamera *webcam* merupakan perangkat *input* visual yang berfungsi untuk menangkap citra atau video dan mengirimkannya ke komputer melalui antarmuka USB atau jaringan (Sutarti et al., 2022). *Webcam* umumnya dilengkapi dengan sensor gambar berbasis CMOS atau CCD yang mampu mengubah cahaya menjadi sinyal digital sehingga dapat diproses perangkat lunak.

Integrasi *webcam* dengan perangkat lunak berbasis *computer vision* (seperti OpenCV, dan YOLO) memungkinkan sistem untuk melakukan analisis visual. Dengan demikian, *webcam* berfungsi sebagai sensor penting dalam implementasi sistem, *smart surveillance*, dan robotika industri. Pada Gambar 2.3 ditampilkan jenis kamera yang digunakan dalam penelitian ini, yaitu Logitech C920e. Kamera ini merupakan salah satu jenis *webcam* populer yang memiliki kualitas tinggi dengan dukungan resolusi hingga Full HD 1080p, sehingga mampu menangkap gambar dengan detail yang baik.



Gambar 2. 3. Kamera Logitech C920e
(Sumber: logitech.com)

Dalam penelitian ini, kamera Logitech C920 dipilih karena sangat handal dalam aplikasi visi komputer, terutama dalam deteksi dan pengenalan objek berbasis algoritma *computer vision* seperti YOLO. Selain itu, kemampuan merekam pada kecepatan *frame* yang cukup tinggi membantu proses deteksi.

E. *Computer Vision*

Computer vision adalah cabang ilmu kecerdasan buatan (AI) yang berfokus pada bagaimana mesin dapat "melihat" dan menafsirkan informasi visual dari dunia nyata melalui gambar dan video (Permana et al., 2025). Tujuan utama dari *computer vision* adalah untuk memberi komputer kemampuan untuk memahami, menganalisis, dan mengekstrak informasi penting dari data visual

seperti yang dilihat oleh orang. Akuisisi gambar, pra-pemrosesan, ekstraksi fitur, dan interpretasi adalah semua bagian dari proses ini.

Perkembangan metode pengolahan gambar dan pembelajaran mesin sangat mendukung *computer vision* dalam aplikasinya. Misalnya, metode *image preprocessing* meningkatkan kualitas gambar. Di sisi lain, algoritma berbasis *deep learning* seperti *Convolutional Neural Network* (CNN) sangat penting untuk klasifikasi, segmentasi, dan deteksi objek (Sari, 2024). Sistem pengenalan wajah, kendaraan otonom, inspeksi kualitas produk di industri, dan integrasi dengan robot cerdas untuk mendukung otomatisasi proses produksi adalah beberapa contoh aplikasi *computer vision* yang semakin meningkat. *Computer vision* adalah teknologi penting untuk mengembangkan sistem berbasis *vision* yang efisien dan presisi.

F. YOLO (*You Look Only Once*)

YOLO (*You Only Look Once*) adalah algoritma deteksi objek berbasis *deep learning* yang dirancang untuk mendeteksi objek dengan cepat dan akurat (Firdaus, 2025). YOLO mengintegrasikan keduanya dalam satu *Convolutional Neural Network* (CNN). Ini memungkinkan YOLO untuk mendeteksi objek dalam satu proses (*single forward pass*) pada gambar atau video, yang menghasilkan performa *real-time* dengan kecepatan yang lebih tinggi dibandingkan dengan metode lain seperti R-CNN, *Fast R-CNN*, dan *Faster R-CNN*. Ini berbeda dengan metode deteksi objek konvensional yang memisahkan proses ekstraksi fitur dan klasifikasi. Pada Gambar 2.4 ditunjukkan contoh penerapan *bounding box* pada hasil deteksi objek menggunakan algoritma YOLO.



Gambar 2. 4. *Bounding box* dengan YOLO
(Sumber: (Diwan et al., 2022))

Pada penelitian ini digunakan metode YOLO26 (*You Only Look Once*) sebagai model utama untuk sistem deteksi objek yang berfungsi menentukan posisi koordinat objek (*workpiece*) secara *real-time*. YOLO26 dirilis pada September 2025 oleh Ultralytics dan merupakan versi YOLO paling mutakhir yang dioptimalkan untuk perangkat *edge* dan inferensi *real-time*. Arsitektur YOLO26 dibangun di atas tiga prinsip utama, yaitu kesederhanaan, efisiensi *deployment*, dan inovasi pelatihan.

Secara struktural, YOLO26 terdiri dari tiga komponen utama, yaitu *Input*, *Backbone*, *Neck*, dan *Head*. Pada bagian *Input*, citra berukuran 640×640 piksel dimasukkan ke dalam model dan dilakukan proses *preprocessing* berupa *resize* dan normalisasi, dilanjutkan *data augmentation* seperti *scaling*, *rotation*, dan *flipping* untuk meningkatkan kemampuan generalisasi model. Berbeda dengan YOLOv5 yang menggunakan *anchor box*, YOLO26 tidak menghapus *Distribution Focal Loss* (DFL) sehingga proses regresi *bounding box* menjadi lebih ringan dan kompatibel dengan berbagai *platform hardware* (ONNX, TensorRT, CoreML).

Setiap prediksi menghasilkan koordinat *bounding box* (x, y, w, h), skor kepercayaan (*confidence score*), dan probabilitas kelas (*class probability*). Koordinat *bounding box* dihitung berdasarkan persamaan regresi:

$$b_x = \sigma(t_x) + c_x \quad (1)$$

$$b_y = \sigma(t_y) + c_y \quad (2)$$

$$b_w = p_w e^{t_w} \quad (3)$$

$$b_h = p_h e^{t_h} \quad (4)$$

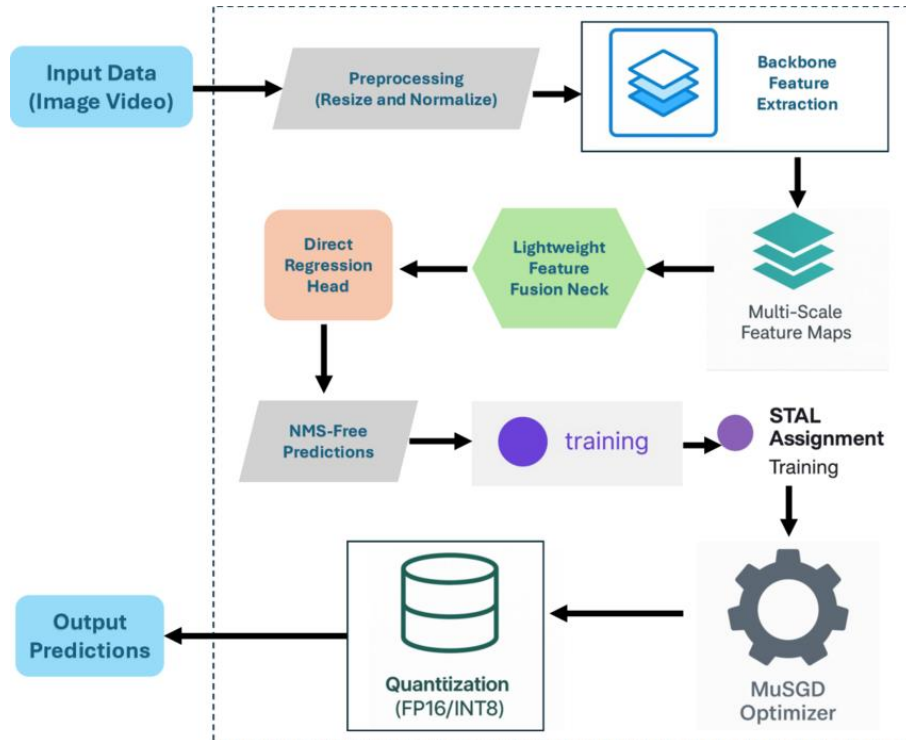
Dengan (t_x, t_y, t_w, t_h) adalah hasil keluaran jaringan, (c_x, c_y) posisi grid cell, dan (p_w, p_h) posisi objek. Selama proses pelatihan, YOLO26 memperkenalkan dua inovasi utama. Pertama, *Progressive Loss Balancing* (ProgLoss) yang menstabilkan pelatihan dan meningkatkan akurasi secara bertahap. Kedua, *Small-Target-Aware Label Assignment* (STAL) yang secara khusus meningkatkan kemampuan deteksi objek berukuran kecil. Selain itu, YOLO26 menggunakan optimizer baru bernama *MuSGD* — hibrida antara

SGD dan Muon yang diadaptasi dari kemajuan pelatihan model bahasa besar untuk konvergensi yang lebih stabil. Fungsi *loss* total gabungan YOLO26 tetap mencakup tiga komponen: *Localization loss* (L_1) untuk kesalahan posisi bounding box, *Confidence loss* (L_2) untuk keyakinan keberadaan objek, dan *Classification loss* (L_3) untuk prediksi kelas objek.

Secara matematis, total *loss function* dapat dituliskan sebagai:

$$L_{total} = \lambda_1 L_{loc} + \lambda_2 L_{conf} + \lambda_3 L_{cls} \quad (5)$$

dengan $\lambda_1, \lambda_2, \lambda_3$ merupakan bobot penyeimbang antar komponen *loss*. Proses *gradient backpropagation* kemudian dilakukan untuk memperbarui bobot jaringan berdasarkan nilai *loss* tersebut, sehingga akurasi deteksi meningkat seiring pelatihan. Arsitektur lengkap algoritma YOLO26 dapat dilihat pada gambar 2.5.

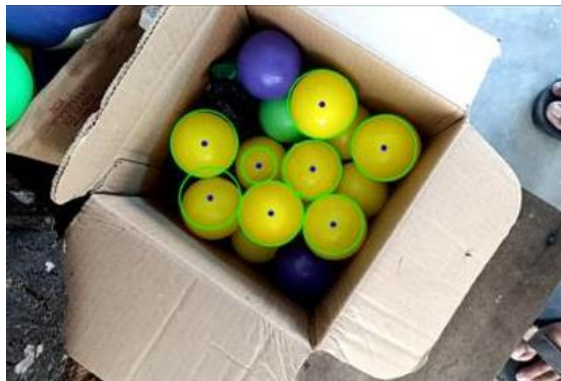


Gambar 2. 5. Arsitektur Yolov26
(Sumber: (Wu et al., 2021))

G. Circle detection

Circle detection adalah teknik penting dalam pengolahan citra komputer yang digunakan untuk mengidentifikasi dan mengekstrak objek berbentuk lingkaran dari citra digital (Rahmadhany & Prasetio, 2025). Teknik ini

memainkan peran penting dalam berbagai aplikasi industri, seperti inspeksi kualitas produk, deteksi komponen dalam perakitan elektronik, sistem navigasi robot, dan proses *pick-and-place* yang memerlukan penempatan presisi objek berbentuk lingkaran. Deteksi lingkaran dilakukan melalui tahap-tahap pengolahan citra, mulai dari segmentasi warna, penghilangan *noise*, hingga analisis bentuk, untuk memastikan bahwa objek yang terdeteksi sesuai dengan karakteristik geometris lingkaran. Pada Gambar 2.6 memperlihatkan bagaimana sistem *computer vision* mampu mengenali serta menandai objek berbentuk lingkaran secara otomatis dari citra masukan.



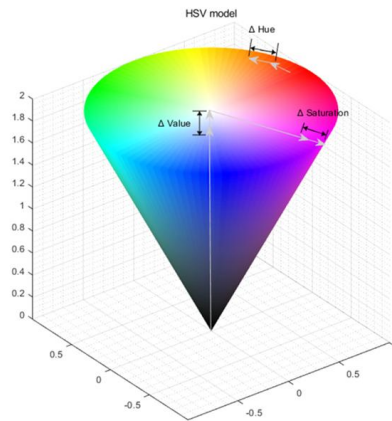
Gambar 2. 6. Deteksi lingkaran
(Sumber : (Hikmahwan et al., 2024))

1. Metode *Hue, saturation, and Value* (HSV)

Metode HSV adalah pendekatan yang populer dalam pemrosesan gambar digital yang digunakan untuk segmentasi berdasarkan warna (Gafari et al., 2025). HSV adalah ruang warna yang mewakili warna dalam tiga komponen: *Hue*, *Saturation*, dan *Value*. *Hue* mewakili warna dasar (misalnya merah, hijau, biru) yang diungkapkan dalam derajat 0–360°, *Saturation* menggambarkan tingkat saturasi atau kemurnian warna, sementara *Value* mewakili kecerahan atau intensitas cahaya warna. Dengan memisahkan ketiga komponen ini, HSV memberikan representasi warna yang lebih intuitif dibandingkan dengan ruang warna RGB.

Representasi ruang warna HSV dapat dilihat pada gambar 2.7 dalam bentuk model kerucut. Model ini menggambarkan tiga komponen utama warna: *Hue*, *Saturation*, dan *Value*. *Hue* ditampilkan pada lingkaran

horizontal di bagian atas, yang mewakili nuansa warna dari 0° hingga 360° , seperti merah, hijau, biru, dan sebagainya. *Saturation* dijelaskan sebagai jarak radial dari pusat ke tepi lingkaran, di mana semakin jauh dari pusat berarti warna semakin jenuh atau intens. Sementara itu, *Value* ditampilkan pada sumbu vertikal, di mana nilai yang lebih tinggi berarti warna yang lebih cerah, dan nilai yang lebih rendah berarti mendekati hitam.



Gambar 2. 7. Model HSV
(Sumber: (Zhong & Wang, 2022))

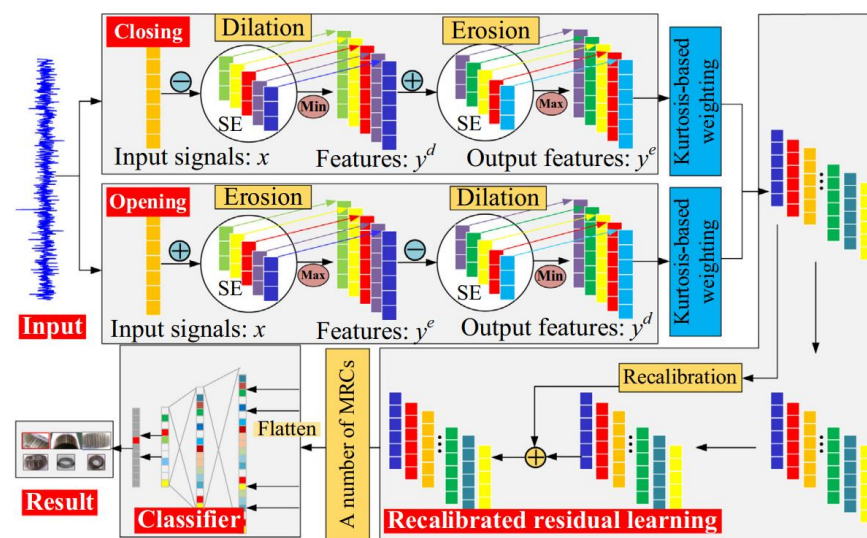
2. *Morphological Operation*

Morphological Operation merupakan transformasi *nonlinier* yang ampuh untuk ekstraksi fitur yang mempertahankan karakteristik penting gambar, seperti batas, bentuk, dan informasi struktural (Roy et al., 2021). Tujuan utama dari *Morphological Operation* adalah untuk memperbaiki hasil segmentasi, menghilangkan gangguan (*noise*), menonjolkan fitur tertentu, serta mempertahankan bentuk dasar objek.

Dalam penerapan *morphological operation*, terdapat beberapa teknik dasar yang digunakan untuk memanipulasi bentuk dan struktur objek pada citra digital. Setiap teknik memiliki fungsi spesifik sesuai dengan kebutuhan pemrosesan citra, terutama dalam meningkatkan kualitas hasil segmentasi dan memperbaiki bentuk objek. Operasi-operasi tersebut meliputi erosi, dilatasi, pembukaan (*opening*), dan penutupan (*closing*).

Erosi berfungsi mengikis batas luar objek sehingga ukuran objek berkurang, berguna untuk menghilangkan *noise* kecil yang tidak diinginkan.

Sebaliknya, dilatasi memperluas batas luar objek sehingga ukurannya bertambah, bermanfaat untuk menutup celah kecil di dalam objek. Pembukaan (*opening*) merupakan kombinasi antara erosi yang dilanjutkan dengan dilatasi, digunakan untuk menghilangkan *noise* sambil tetap mempertahankan bentuk utama objek. Sementara itu, penutupan (*closing*) merupakan kebalikan dari *opening*, yaitu kombinasi dilatasi kemudian erosi, berfungsi untuk menutup lubang atau celah kecil pada objek. Seluruh proses tersebut dapat diamati pada gambar 2.8, yang memperlihatkan bagaimana *morphological operation* diterapkan.



Gambar 2. 8. *Morphological Convolutional Neural Network Process*
(Sumber: (Ye & Yu, 2021))

3. Roundness Filter

Roundness filter adalah metode analisis bentuk yang digunakan untuk menilai sejauh mana suatu objek menyerupai lingkaran sempurna. Konsep ini sering digunakan dalam pengolahan citra komputer setelah segmentasi dan penyaringan *noise*, dengan tujuan memastikan bahwa hanya objek yang benar-benar menyerupai lingkaran yang dipertahankan untuk analisis lebih lanjut.

Roundness biasanya didefinisikan sebagai rasio antara luas suatu objek dengan kuadrat kelilingnya, atau melalui parameter *roundness ratio*. Nilai *roundness* yang mendekati 1 menunjukkan objek yang sempurna berbentuk

lingkaran, sementara nilai yang lebih kecil menunjukkan bentuk yang semakin jauh dari lingkaran. Untuk mengetahui tingkat *roundness* menggunakan persamaan berikut:

$$Roundness = \frac{4\pi \times Area}{(Perimeter)^2} \quad (6)$$

Dimana:

Roundness : Tingkat kebulatan ≥ 0.75

Area : Luas area piksel diarea *contour* (piksel)

Perimeter : Keliling *contour* (piksel)

Untuk mengetahui luas area digunakan persamaan berikut:

$$Area = \sum_{(x,y) \in contour} 1 \quad (7)$$

Dimana:

x,y : Koordinat titik piksel dalam diagram kartesius posisi *contour*

$$Perimeter = \sum_{i=1}^n d(P_i, P_{i+1}) \quad (8)$$

$$d(P_i, P_{i+1}) = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} \quad (9)$$

Dimana:

P : Titik ke i pada *contour*

$d(P_i, P_{i+1})$: Fungsi jarak titik P_i dan P_{i+1}

H. *Vision Guide Robotics*

Vision Guided Robotics (VGR) adalah sistem robotika yang mengintegrasikan teknologi penglihatan komputer (*computer vision*) dengan mekanisme pengendalian robot untuk menciptakan sistem yang cerdas, adaptif, dan presisi tinggi. Dengan sistem ini, robot tidak hanya beroperasi berdasarkan perintah statis yang telah diprogram sebelumnya, tetapi juga mampu merespons kondisi lingkungan secara dinamis melalui masukan visual yang ditangkap oleh kamera atau sensor optik. Teknologi ini sering diibaratkan sebagai "mata" robot, memungkinkan robot untuk mengenali objek, menganalisis posisi dan orientasinya, serta mengambil keputusan secara otomatis berdasarkan informasi visual yang diterimanya secara *real-time*.

Menurut Singh et al. (2022), *vision guided robotic systems* merupakan sistem yang menggabungkan kemampuan persepsi visual dengan algoritma kendali cerdas untuk memungkinkan robot beradaptasi terhadap perubahan lingkungan kerja yang tidak terstruktur. Sistem ini tidak hanya meningkatkan fleksibilitas operasional robot, tetapi juga secara signifikan mengurangi ketergantungan pada *fixture* dan *jig* yang mahal serta proses pemrograman *teaching point* yang memakan waktu. Implementasi VGR dapat meningkatkan fleksibilitas sistem robotik hingga 70% dibandingkan sistem konvensional berbasis koordinat tetap, menjadikannya solusi yang sangat relevan untuk kebutuhan *mass customization* dalam industri manufaktur modern.

Robinson et al. (2023) menegaskan bahwa integrasi teknologi *robotic vision* dalam sistem manufaktur modern bukan hanya sekadar peningkatan teknologi, melainkan merupakan perubahan paradigma fundamental dalam cara robot berinteraksi dengan lingkungannya. Sistem VGR yang canggih mampu melakukan kalibrasi posisi secara dinamis, menangani variasi produk tanpa pemrograman ulang, serta beradaptasi terhadap perubahan kondisi seperti perbedaan pencahayaan, variasi ukuran objek, dan ketidakberaturan posisi *workpiece* di atas *conveyor* atau pallet. Kemampuan adaptif ini menjadikan VGR sebagai komponen kritis dalam implementasi *smart manufacturing* dan otomasi industri yang fleksibel.

I. TCP/IP Communication

TCP/IP (*Transmission Control Protocol/Internet Protocol*) adalah kumpulan protokol komunikasi standar yang digunakan dalam jaringan komputer, termasuk sistem otomatisasi industri. TCP/IP mendefinisikan cara data dikemas, dialamatkan, dikirim, diterima, dan diinterpretasikan antara perangkat yang terhubung ke jaringan. Keunggulan utamanya adalah keandalan, fleksibilitas, dan skalabilitas, sehingga membuatnya sangat populer untuk menghubungkan sistem heterogen, termasuk komputer, robot industri, PLC, dan perangkat IoT.

Dalam sistem otomatisasi, TCP/IP memainkan peran krusial dalam menyediakan komunikasi *real-time* antara perangkat pengendali

(pengendali/PC) dan aktuator (robot, sensor, mesin produksi). Protokol ini memungkinkan data seperti koordinat posisi, perintah gerakan, dan hasil deteksi penglihatan komputer untuk dikirimkan dengan cepat dan akurat.

Terdapat berbagai standar dan varian protokol komunikasi berbasis TCP/IP yang dikembangkan secara khusus untuk kebutuhan industri, masing-masing dengan karakteristik *latency*, kecepatan transfer data, dan domain aplikasi yang berbeda. Standar-standar tersebut disajikan secara terstruktur pada Tabel 2.X berikut untuk memberikan gambaran komprehensif mengenai posisi dan kesesuaian protokol TCP/IP yang digunakan dalam penelitian ini dibandingkan dengan standar industri yang berlaku.

Tabel 2. 2. Standar Komunikasi TCP/IP Industri

No	Standar Protokol	Organisasi	Batas Latency	Keterangan
1	IEEE 802.3 (<i>Ethernet</i> Industri)	IEEE	≤ 100 ms	Standar <i>Ethernet</i> umum untuk jaringan industri berbasis kabel UTP/Cat5e–Cat6
2	Profinet RT (<i>Real-Time</i>)	Profibus & <i>Profinet</i> <i>International</i> (PI)	≤ 10 ms	Standar komunikasi <i>real-time</i> industri berbasis <i>Ethernet</i> untuk kendali mesin dan robot 100 Mbps
3	Modbus TCP	<i>Modbus</i> <i>Organization</i>	≤ 100 ms	Protokol sederhana berbasis TCP/IP untuk komunikasi perangkat industri seperti sensor dan aktuator
4	Protokol sederhana berbasis TCP/IP	OPC <i>Foundation</i>	≤ 50 ms	Standar komunikasi <i>machine-to-machine</i> berbasis TCP/IP untuk

	untuk komunikasi perangkat industri seperti sensor dan aktuator			sistem otomasi dan Industri 4.0
5	TCP/IP Socket (<i>Standard</i>)	IETF (RFC 793)	≤ 50 ms	Komunikasi berbasis <i>socket</i> Python standar yang digunakan pada sistem VGR dalam penelitian ini

Untuk melakukan komunikasi TCP/IP Robot Scara T3 dengan python maka digunakan beberapa komponen berikut:

1. Kabel *Ethernet*

Kabel *Ethernet* merupakan media transmisi utama yang digunakan untuk menghubungkan robot SCARA T3 ke komputer atau pengontrol melalui protokol TCP/IP. Kabel ini berfungsi sebagai saluran komunikasi data dua arah, memungkinkan pertukaran informasi berupa perintah kontrol, koordinat posisi, dan umpan balik status robot secara *real-time*.



Gambar 2. 9. Kabel *Ethernet*
(Sumber: id.cat6acabling.com)

2. Konverter *Ethernet to Type C*

Konverter *Ethernet to Type-C* merupakan perangkat tambahan yang digunakan untuk menghubungkan kabel *Ethernet* ke komputer atau laptop yang tidak memiliki *port* LAN bawaan. Perangkat ini berfungsi sebagai adapter jaringan, yang mengubah sinyal dari konektor RJ-45 (*Ethernet*)

menjadi sinyal yang dapat diterima melalui *port* USB *Type-C* pada komputer *modern*.



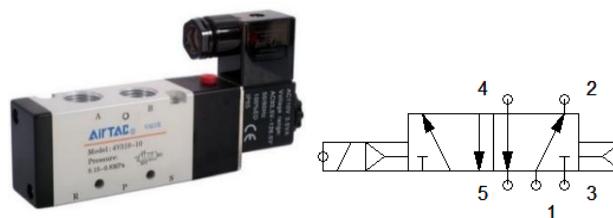
Gambar 2. 10. Konverter *Ethernet to Type C*
(Sumber: quinton.id)

J. Sistem *Pneumatik*

Sistem *pneumatik* adalah sistem yang menggunakan udara terkompresi sebagai media untuk menghasilkan gerakan mekanis atau mengoperasikan aktuator. Prinsip kerja *pneumatik* didasarkan pada hukum fisika fluida, khususnya Hukum Pascal dan Hukum Boyle, di mana udara terkompresi dapat menyimpan energi yang kemudian dilepaskan untuk menggerakkan komponen-komponen tertentu seperti silinder, aktuator, dan peralatan otomatisasi.

1. Selenoid *valve*

Selenoid *valve* adalah katup yang dikendalikan secara elektrik yang menggunakan kumparan elektromagnetik (solenoid) untuk membuka atau menutup aliran udara bertekanan. Komponen ini berfungsi sebagai katup pengatur arah dalam sistem pneumatik, memungkinkan udara diarahkan ke aktuator atau dilepaskan ke atmosfer. Katup solenoid tersedia dalam beberapa konfigurasi, seperti *2/2-way*, *3/2-way*, dan *5/2-way*, yang digunakan sesuai dengan kebutuhan sistem. Namun pada perancangan kali ini digunakan selenoid *valve* *5/2* yang ditunjukkan pada gambar 11 beserta simbol.



Gambar 2. 11. Bentuk dan simbol Selenoid *valve*
(Sumber: us-en.airtac.com)

2. *Air Filter Regulator*

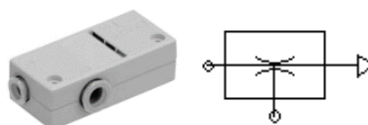
Air Filter Regulator adalah komponen yang menyaring udara terkompresi dari debu dan kelembapan sambil secara bersamaan mengatur tekanan untuk menjaga kestabilan. Filter ini mencegah partikel kotoran masuk ke dalam sistem, yang dapat menyebabkan kerusakan atau keausan pada aktuator dan katup. Sementara itu, regulator menjaga tingkat tekanan yang konstan, memastikan operasi yang konsisten dan aman dari sistem pneumatik. *Air Filter Regulator* biasanya dipasang di awal jalur distribusi udara, memastikan kualitas udara yang masuk ke semua komponen tetap terjaga. Bentuk dan simbol dari *Air Service Regulator* dapat dilihat pada gambar 2.12.



Gambar 2. 12. Bentuk dan simbol *Air Filter Regulator*
(Sumber: us-en.airtac.com)

3. *Vacuum Generator*

Vacuum Generator, atau *Venturi Vacuum*, digunakan untuk menghasilkan tekanan negatif (vakum) dari sumber udara terkompresi. Prinsip kerjanya memanfaatkan efek *Venturi*, yaitu perubahan kecepatan aliran udara yang menyebabkan penurunan tekanan, sehingga membentuk vakum. Komponen ini banyak digunakan dalam aplikasi *pick-and-place*, terutama ketika robot perlu mengambil objek menggunakan cangkir hisap. Keuntungan generator vakum meliputi ukurannya yang kompak, tidak memerlukan motor listrik tambahan, dan kemudahan integrasinya ke dalam sistem pneumatik.



Gambar 2. 13. Bentuk dan simbol *Vacuum Generator*
(Sumber: www.smcpneumatics.com)

4. *Vacuum Suction Cup*

Suction Cup adalah perangkat berbentuk *cup* yang elastis dan menggunakan tekanan negatif dari generator vakum. Komponen ini berfungsi sebagai “jari” atau “penjepit” dalam aplikasi *pick-and-place*, di mana objek ditahan oleh hisapan ke permukaan. *Suction Cup* dapat terbuat dari berbagai bahan seperti karet, silikon, atau poliuretan, yang dipilih sesuai dengan karakteristik objek yang diangkat (halus, kasar, ringan, atau berat). Keuntungan dari *Suction Cup* adalah kemampuannya untuk memindahkan objek tanpa merusak permukaan, fleksibilitasnya dalam menangani berbagai bentuk objek, dan presisi tinggi dalam aplikasi industri. Bentuk dan simbol dari *vacuum suction cup* dapat dilihat pada gambar 2.14.



Gambar 2. 14. Bentuk dan simbol *Vacuum Suction Cup*
(Sumber: www.festo.com)

K. *Power Supply Unit*

Power Supply Unit (PSU) adalah komponen yang menyediakan dan mengatur kebutuhan daya listrik suatu sistem, baik untuk perangkat keras komputer, robot industri, maupun perangkat periferal lainnya. PSU umumnya bekerja dengan mengubah arus bolak-balik (AC) dari jaringan listrik menjadi arus searah (DC) yang lebih stabil dan sesuai dengan kebutuhan beban. Stabilitas daya sangat berpengaruh terhadap performa sistem, sebab gangguan daya seperti, fluktuasi, atau *noise* dapat menyebabkan *error*, bahkan kerusakan komponen elektronik. *Power Supply Unit* yang digunakan dari Omron yang dapat dilihat pada gambar 2.15.



Gambar 2. 15. Omron *Power Supply Unit*
(Sumber: www.omron.co.id)

L. Python

Python adalah bahasa pemrograman tingkat tinggi yang dikembangkan oleh Guido van Rossum pada tahun 1991. Dirancang dengan filosofi keterbacaan, fleksibilitas, dan dukungan komunitas yang luas, Python telah menjadi salah satu bahasa pemrograman paling populer di dunia. Python adalah bahasa pemrograman yang diinterpretasikan, tingkat tinggi, serbaguna, dan open-source, sehingga banyak digunakan dalam otomatisasi industri, kecerdasan buatan (AI), pembelajaran mesin, penglihatan komputer, dan integrasi dengan sistem robotika.

Keunggulan utama Python adalah sintaksnya yang sederhana dan mudah dipahami, bahkan bagi pemula. Selain itu, Python memiliki beragam perpustakaan dan kerangka kerja, yang memudahkan pengembangan aplikasi kompleks dalam waktu yang lebih singkat. Python berfungsi sebagai jembatan antara sistem penglihatan (YOLO) dan pengendali robot melalui komunikasi TCP/IP. Python mengambil data koordinat dari deteksi objek, memprosesnya, dan kemudian mengirim instruksi ke robot untuk melakukan *gerakan pick-and-place* secara *real-time*. Dengan dukungan perpustakaan yang kuat, Python dapat mengintegrasikan berbagai komponen seperti kamera, jaringan, dan kontrol aktuator menjadi sistem yang terkoordinasi secara tunggal.

M. General User Interface

General User Interface (GUI) adalah antarmuka grafis yang memungkinkan pengguna berinteraksi dengan perangkat lunak atau sistem komputer melalui elemen visual seperti tombol, menu, ikon, jendela, maupun grafik. GUI dikembangkan untuk menggantikan *Command Line Interface* (CLI) yang berbasis teks, sehingga sistem menjadi lebih *user-friendly* dan mudah digunakan oleh pengguna dari berbagai latar belakang, termasuk yang tidak memiliki pengetahuan teknis mendalam.

N. Penelitian Terdahulu

Beberapa penelitian terdahulu yang menjadi dasar pemilihan metode dan algoritma dalam penelitian ini adalah sebagai berikut:

1. (Kivrak & Gurbuz, 2022) dengan judul penelitian *Performance Comparison of YOLOv3, YOLOv4, YOLOv5, Algorithms: A Case Study for Poultry Recognition*. Penelitian ini menunjukkan bahwa integrasi algoritma YOLO dengan sistem robotik mampu meningkatkan efisiensi proses produksi. Hasil penelitian tersebut menyimpulkan bahwa YOLOv5 dibandingkan dengan YOLOv3 dan v4 memiliki *trade-off* terbaik antara akurasi dan kecepatan, dan YOLO26 sebagai versi terbaru hadir dengan peningkatan lebih lanjut berupa inferensi *end-to-end* tanpa NMS serta optimasi untuk perangkat edge. Penelitian ini menjadi landasan pemilihan YOLO26 dalam penelitian ini. Hasil awal penelitian Kivrak & Gurbuz menyimpulkan bahwa YOLOv5 dibandingkan dengan YOLOv3 dan v4 memiliki *trade-off* terbaik antara akurasi dan kecepatan, sehingga sangat sesuai untuk aplikasi *real-time* pada sistem robot industri.
2. (Bochkovskiy et al., 2020) dengan judul penelitian *YOLOv4: Optimal Speed and Accuracy of Object Detection*. Penelitian ini mengkonfirmasi bahwa algoritma YOLO memiliki performa yang lebih unggul dibandingkan model deteksi objek lain seperti Faster R-CNN dan SSD pada lingkungan industri yang membutuhkan kecepatan inferensi tinggi. Keunggulan YOLO terletak pada kemampuannya melakukan deteksi objek secara *real-time* dengan latensi rendah.
3. (Zhao et al., 2022) dengan judul penelitian *Palletizing Robot Positioning Bolt Detection Based on Improved YOLO-V3*. Penelitian ini mengusulkan metode deteksi baut untuk sistem robot *palletizing* berbasis algoritma *Improved YOLO-V3*. YOLO-V3 dimodifikasi untuk meningkatkan akurasi deteksi objek berukuran kecil dan memiliki kompleksitas tinggi. Hasil eksperimen menunjukkan bahwa metode tersebut mampu mengungguli model deteksi objek populer lainnya, seperti SSD dan Faster R-CNN, pada *dataset palletizing* baut.
4. Penelitian (Sapkota & Karkee, 2025) dengan judul penelitian *Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8, and YOLOv5 Object Detectors for Computer Vision and Pattern Recognition*.

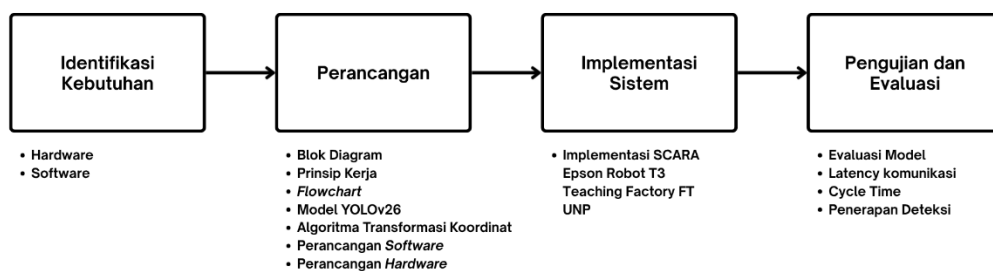
Penelitian ini membahas perkembangan arsitektur algoritma YOLO dari YOLOv5 hingga YOLO26 dengan fokus pada peningkatan akurasi, efisiensi, dan kemudahan *deployment* pada berbagai perangkat. YOLO26 sebagai versi terbaru mengusulkan beberapa inovasi utama seperti penghapusan *Distribution Focal Loss* (DFL), penerapan *NMS-free inference*, serta penggunaan metode *Progressive Loss Balancing* (ProgLoss) dan *Small-Target-Aware Label Assignment* (STAL) untuk meningkatkan stabilitas pelatihan dan akurasi deteksi objek kecil. Hasil benchmarking pada *dataset* MS COCO menunjukkan bahwa YOLO26 mampu mencapai akurasi yang lebih tinggi dengan waktu inferensi yang lebih cepat dibandingkan versi sebelumnya seperti YOLOv5, YOLOv8, dan YOLO11, sehingga lebih optimal untuk implementasi *real-time* pada perangkat *edge*.

BAB III

METODE PERANCANGAN

Proses perancangan tugas akhir ini menggunakan metode perancangan *Engineering Design Process* (EDP), yang terdiri dari beberapa tahap sistematis untuk merancang solusi teknis yang optimal. *Engineering Design Process* (EDP) adalah suatu pendekatan sistematis dan berurutan yang digunakan dalam rekayasa untuk memecahkan masalah teknik atau menciptakan solusi desain yang efektif dan efisien (Aini et al., 2025). Tahap-tahap ini meliputi Identifikasi Kebutuhan (*Hardware* dan *Software*) Spesifikasi Teknis, Perancangan, Implementasi dengan sistem, Pengujian dan Evaluasi.

Tahapan dari *Engineering Design Process* divisualisasikan dalam gambar berikut:



Gambar 3. 1. Metode *Engineering Design Process*

A. Identifikasi Kebutuhan

Dalam perancangan sistem *object detection* berbasis YOLO untuk kontrol robot SCARA T3 pada proses *pick and place*, penting untuk menentukan dan menganalisis kebutuhan perangkat keras (*hardware*) dan perangkat lunak (*software*) yang dibutuhkan untuk mendukung kelancaran dan efektivitas sistem. Berikut adalah analisis kebutuhan *hardware* dan *software* yang diperlukan untuk mendukung implementasi sistem ini:

1. *Hardware*

Hardware yang dibutuhkan untuk mendukung sistem *object detection* berbasis YOLO serta integrasi dengan robot SCARA T3 tidak hanya terbatas pada perangkat utama, tetapi juga mencakup komponen pendukung yang memastikan keseluruhan sistem dapat bekerja secara stabil.

a. Robot Scara T3 B401S

Robot SCARA T3 B401S digunakan sebagai aktuator utama dalam sistem *pick and place*. Robot ini memiliki karakteristik kecepatan tinggi dan akurasi yang baik sehingga sesuai untuk aplikasi pemindahan objek secara berulang. Robot SCARA T3 dikendalikan melalui sistem kontrol berbasis komputer dengan komunikasi Ethernet, yang memungkinkan integrasi dengan sistem vision.

b. Kamera Logitech C920

Kamera Logitech C920 digunakan sebagai sensor visual untuk menangkap citra objek yang akan diproses oleh algoritma YOLO. Kamera ini memiliki resolusi *Full HD* yang cukup untuk mendukung proses *object detection* dengan tingkat akurasi yang baik.

c. PC/Laptop

PC atau laptop digunakan sebagai pusat pemrosesan sistem, yang bertugas menjalankan algoritma YOLO, memproses citra dari kamera, melakukan perhitungan koordinat objek, serta mengirimkan data posisi ke robot SCARA.

2. *Software*

Perangkat lunak yang digunakan dalam sistem ini meliputi sistem operasi, perangkat lunak untuk pemrograman dan pelatihan model YOLO, serta *software* untuk kontrol robot SCARA. Berikut adalah perangkat lunak yang dibutuhkan:

a. YOLO26

YOLO26 dipilih sebagai algoritma *object detection* dalam tugas akhir ini karena memiliki keseimbangan yang baik antara kecepatan, akurasi, dan kemudahan implementasi, sehingga sesuai untuk aplikasi *vision-guided pick and place* pada robot SCARA T3. Dibandingkan versi YOLO sebelumnya, YOLO26 memiliki tingkat stabilitas yang tinggi serta unggul dalam tiga aspek kunci: (1) inferensi end-to-end tanpa NMS yang mengurangi latensi pipeline, (2) arsitektur yang lebih

ringan berkat penghapusan *Distribution Focal Loss* (DFL) sehingga mudah di-*deploy* ke berbagai *hardware* termasuk CPU dan perangkat edge, dan (3) akurasi deteksi objek kecil yang lebih baik berkat STAL.

Selain itu, YOLO26 dikembangkan berbasis *framework* PyTorch yang mendukung pengembangan berbasis Python dan OpenCV, sehingga lebih fleksibel dan mudah diintegrasikan dengan sistem kontrol robot. YOLO26 juga mampu melakukan deteksi objek secara *real-time* dengan latensi rendah serta menyediakan beberapa varian model yang dapat disesuaikan dengan kemampuan perangkat keras.

b. Python

Python digunakan sebagai bahasa pemrograman utama dalam pengembangan sistem. Python berperan dalam pengolahan citra, pemrosesan hasil deteksi YOLO, konversi koordinat objek, serta komunikasi data dengan robot SCARA.

Python didukung oleh beberapa library pendukung, antara lain:

1) OpenCV

OpenCV digunakan untuk proses akuisisi citra dari kamera, pemrosesan citra, serta visualisasi hasil deteksi objek. Library ini berperan dalam pengolahan gambar sebelum dan sesudah proses deteksi menggunakan YOLO.

2) Tkinter

Tkinter digunakan untuk membangun antarmuka pengguna (*Graphical User Interface*). GUI ini berfungsi untuk memudahkan pengguna dalam menjalankan sistem, memantau hasil deteksi, serta melakukan kontrol dasar terhadap proses *pick and place*.

3) NumPy

NumPy digunakan untuk pengolahan data numerik, khususnya dalam perhitungan koordinat objek hasil deteksi dan konversinya ke sistem koordinat robot SCARA.

c. Epson RC+

Epson RC+ merupakan *software* resmi yang digunakan untuk pemrograman dan pengendalian robot SCARA maupun 6 Axis dari Epson. *Software* ini digunakan untuk menerima data koordinat dari sistem *vision* via TCP/IP dan mengeksekusi pergerakan robot sesuai dengan perintah yang dikirimkan dari PC/laptop.

d. Roboflow

Roboflow digunakan sebagai platform untuk manajemen *dataset object detection*, meliputi proses anotasi data, augmentasi dataset, serta konversi format *dataset* agar sesuai dengan format YOLO26. Penggunaan Roboflow mempermudah proses pembuatan *dataset* model YOLO.

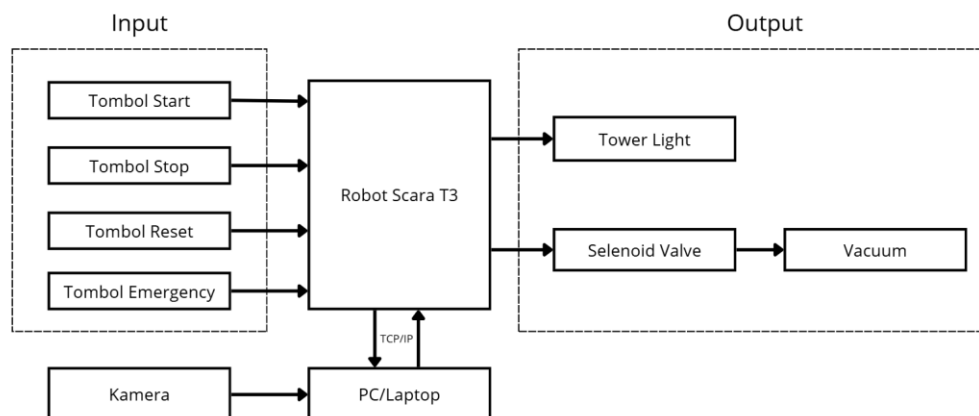
e. Google Collab

Google Colaboratory digunakan sebagai *platform* berbasis *cloud* untuk melakukan pelatihan (*training*) model YOLO26. Platform ini menyediakan sumber daya komputasi yang memadai, seperti GPU, sehingga mempercepat proses pelatihan model tanpa bergantung sepenuhnya pada spesifikasi PC lokal.

B. Perancangan Sistem

1. Blok Diagram

Pada gambar 3.2 merupakan blok diagram yang memiliki tujuan untuk sebagai gambaran dasar mengenai sistem yang dirancang.



Gambar 3. 2. Blok diagram

Sistem ini terdiri dari beberapa komponen utama yang saling terhubung untuk membentuk suatu rangkaian otomasi terintegrasi. Kamera berfungsi sebagai sensor penglihatan yang menangkap citra objek, kemudian hasil tangkapan tersebut diproses pada PC atau laptop menggunakan algoritma deteksi berbasis *computer vision*. Data yang telah diproses selanjutnya dikirimkan ke Robot Scara T3 sebagai referensi pergerakan dalam melakukan tugas *pick and place*.

Selain *input* dari kamera dan PC, robot juga dapat dikendalikan secara manual melalui tombol-tombol kontrol, yaitu tombol *Start* dan *Stop* untuk memulai atau menghentikan operasi, tombol *Reset* untuk menghilangkan status *error* robot, serta tombol *Emergency* yang digunakan untuk menghentikan robot secara cepat dalam keadaan darurat dan mengaktifkan sistem *brake* robot.

Robot Scara T3 juga terhubung dengan aktuator pendukung berupa solenoid *valve* yang mengendalikan *vacuum (suction cup)* sebagai *end-effector* untuk mengambil maupun meletakkan objek. Selain itu, terdapat *tower light* yang berfungsi memberikan informasi visual mengenai status kerja robot, seperti kondisi berjalan normal, berhenti, atau mengalami *error*.

2. Prinsip Kerja

Sistem yang dirancang pada penelitian ini merupakan integrasi antara metode *object detection* berbasis YOLO, pengolahan citra menggunakan OpenCV, serta komunikasi TCP/IP untuk mengendalikan robot SCARA dalam proses *pick and place* secara otomatis. Sistem ini memanfaatkan kamera sebagai sensor utama untuk mendeteksi posisi objek (*workpiece*), sehingga robot mampu bekerja secara fleksibel tanpa bergantung pada posisi tetap.

Proses dimulai dengan akuisisi citra dari kamera, kemudian citra tersebut diproses menggunakan model YOLO untuk mendeteksi objek berupa *pallet*. Setelah objek terdeteksi, sistem membagi area *pallet* menjadi *grid* 3 x 8 untuk *pallet pick* kemudian 12 x 2 untuk *pallet*.

Kemudian, algoritma deteksi lingkaran diterapkan untuk mengidentifikasi keberadaan *workpiece* yang berada pada *grid pallet pick* dan *pallet place*. Proses deteksi dimulai dengan mengubah citra dari ruang warna RGB ke HSV

untuk mempermudah segmentasi warna. Selanjutnya, dilakukan proses *thresholding* untuk mengekstraksi area berwarna putih yang merepresentasikan *workpiece*. Hasil segmentasi kemudian diperbaiki menggunakan operasi morfologi seperti *opening* dan *closing* untuk menghilangkan *noise* serta memperjelas bentuk objek. Setelah itu, sistem melakukan deteksi kontur pada citra hasil segmentasi. Setiap kontur yang ditemukan akan dianalisis berdasarkan luas dan tingkat kebulatannya (*circularity*).

Selanjutnya, sistem menyusun urutan pengambilan objek berdasarkan koordinat terkecil terlebih dahulu, yaitu dimulai dari nilai kolom dan baris paling kecil. Pendekatan ini bertujuan untuk menghasilkan pola pergerakan robot yang terstruktur dan konsisten, serta meminimalkan perpindahan yang tidak perlu.

Pada sisi *pallet* tujuan, yaitu *pallet place*, sistem tidak hanya menerima posisi penempatan, tetapi juga melakukan analisis terhadap kondisi slot yang tersedia. Sistem mendeteksi area yang belum terdapat *workpiece* (slot kosong) berdasarkan hasil pengolahan citra. Slot kosong ini kemudian disusun secara berurutan sebagai target penempatan objek.

Setelah seluruh koordinat *pick* dan slot kosong pada *pallet place* diperoleh, sistem membentuk antrian tugas *pick* and *place*. Setiap tugas berisi pasangan koordinat antara posisi pengambilan dan posisi penempatan.

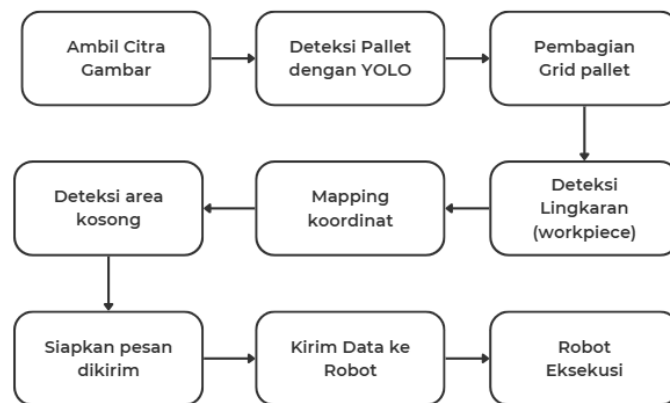
Pada tahap eksekusi, robot akan mulai bekerja setelah seluruh data koordinat selesai diproses dan dikirim. Untuk setiap *workpiece*, robot terlebih dahulu bergerak menuju posisi objek pada *pallet pick*. Setelah mencapai posisi tersebut, suction cup diaktifkan untuk mengambil *workpiece*.

Selanjutnya, robot bergerak menuju *pallet place* ke slot kosong yang telah ditentukan secara berurutan. Pada posisi ini, suction cup dinonaktifkan sehingga *workpiece* dilepaskan dan ditempatkan pada slot yang kosong. Proses ini dilakukan secara berulang hingga seluruh *workpiece* pada *pallet pick* telah dipindahkan.

Setelah semua tugas selesai, robot akan kembali ke posisi awal (*home*) sebagai akhir dari satu siklus kerja. Seluruh proses berlangsung secara otomatis

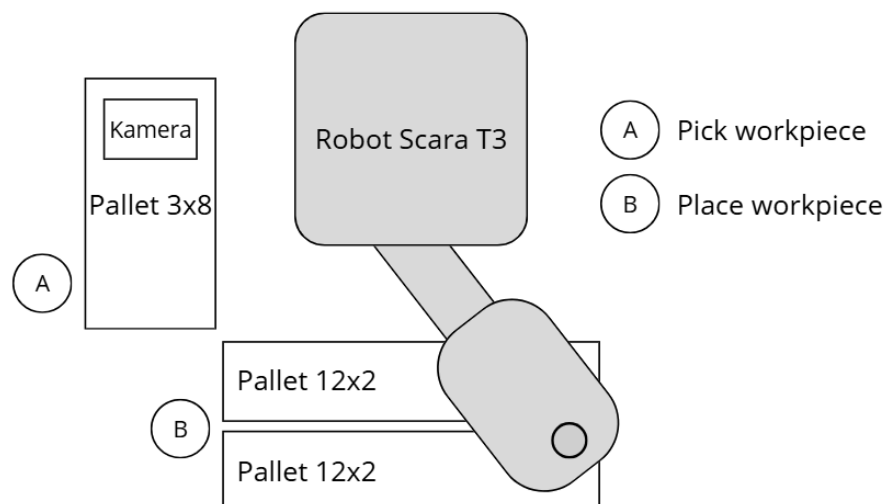
dan dimonitor melalui antarmuka GUI yang menampilkan hasil deteksi, status sistem, serta urutan eksekusi robot secara *real-time*.

Dengan metode ini, sistem mampu melakukan proses *pick and place* secara adaptif terhadap kondisi aktual di lapangan, sekaligus menjaga efisiensi waktu karena proses deteksi dan perencanaan dilakukan sebelum robot mulai bergerak. Untuk memudahkan dalam memahami prinsip kerja sistem ini dapat dipahami dengan melihat gambar 3.3.



Gambar 3. 3. Alur kerja sistem

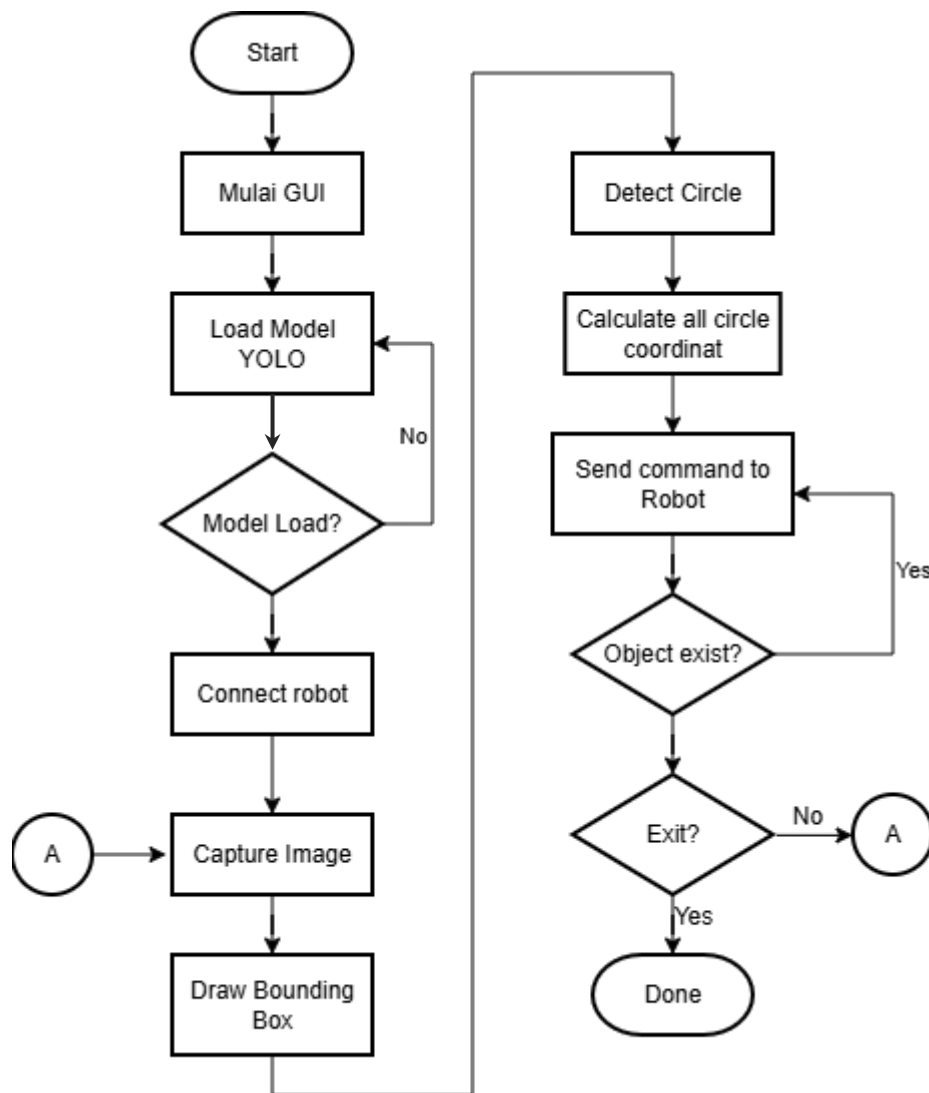
Untuk memberikan gambaran yang lebih jelas mengenai lingkungan operasional sistem, tampilan area kerja ditunjukkan pada gambar berikut.



Gambar 3. 4. Tampilan area

3. Flowchart

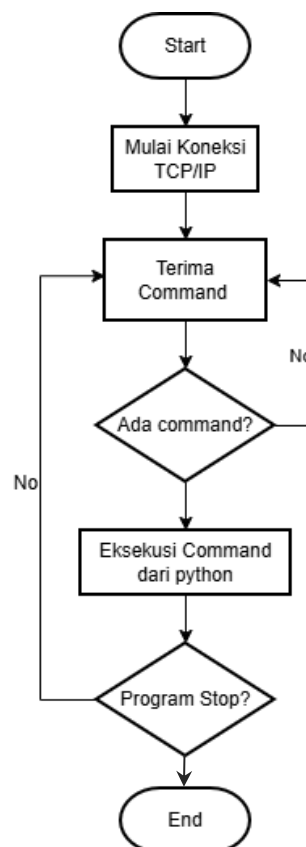
Flowchart memiliki peran yang sangat penting dalam merancang langkah-langkah atau fitur yang akan diimplementasikan dalam sebuah program. Pada gambar 3.5 merupakan *flowchart* dari program python.



Gambar 3. 5. *Flowchart* program python

Flowchart pada gambar 3.5 menggambarkan alur kerja sistem deteksi objek berbasis YOLO yang terintegrasi dengan robot SCARA T3. Proses dimulai dari menjalankan GUI, kemudian sistem memuat model YOLO dan membuka

kamera untuk menangkap citra. Jika model atau kamera belum siap, sistem akan mengulang prosesnya. Setelah objek terdeteksi, koordinat hasil deteksi dikirim ke robot melalui komunikasi TCP/IP untuk dieksekusi sistem akan menunggu perintah “*done*” dari robot agar mengirim koordinat berikutnya ke robot. Siklus ini berlangsung terus hingga pengguna memilih keluar dari program. Dan berikut *flowchart* dari program RC+ dapat dilihat pada gambar 3.6.



Gambar 3. 6. *Flowchart* program RC+

Flowchart di atas menggambarkan alur komunikasi antara sistem Python dengan robot melalui protokol TCP/IP. Proses diawali dari kondisi *Start*, yang menandakan sistem mulai dijalankan. Tahap pertama adalah inisialisasi koneksi TCP/IP, di mana sistem Python melakukan proses koneksi ke robot menggunakan alamat IP dan *port* yang telah ditentukan. Keberhasilan tahap ini menjadi syarat utama agar komunikasi data dapat berlangsung.

Setelah koneksi berhasil, sistem masuk ke tahap penerimaan *command* (*Terima Command*). Pada tahap ini, sistem berada dalam kondisi siap menerima

perintah yang dikirim, baik dari antarmuka pengguna maupun dari proses internal program.

Selanjutnya dilakukan proses pengecekan keberadaan *command* melalui percabangan “*Ada command?*”. Jika tidak terdapat *command* yang masuk, maka sistem akan tetap berada dalam kondisi menunggu (*looping*) hingga *command* tersedia. Apabila *command* diterima, sistem akan melanjutkan ke tahap eksekusi *command* dari Python. Pada tahap ini, perintah yang telah diterima akan dikirim ke robot melalui koneksi TCP/IP dan dijalankan sesuai instruksi, seperti pergerakan robot atau kontrol aktuator.

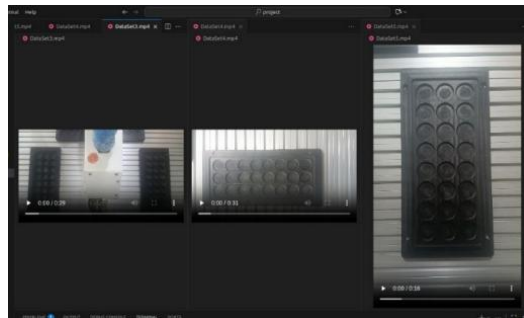
Setelah eksekusi dilakukan, sistem akan melakukan pengecekan kondisi “*Program Stop?*”. Jika program belum dihentikan, maka alur akan kembali ke tahap penerimaan *command* untuk memproses perintah berikutnya secara berulang. Namun, jika kondisi *stop* terpenuhi, maka sistem akan mengakhiri proses.

4. Perancangan Model YOLO26

Perancangan model merupakan langkah krusial dalam pengembangan sistem *pick-and-place* berbasis *computer vision*. Model yang dikembangkan dalam studi ini mengintegrasikan sistem kamera, algoritma *deep learning* YOLO26, metode pemrosesan gambar berbasis HSV, operasi morfologis, dan filter geometris untuk menentukan bentuk dan posisi objek (lingkaran). Perancangan model dijelaskan dalam beberapa tahapan sebagai berikut:

a. Pengambilan *dataset*

Tahap awal dalam proses perancangan model deteksi objek adalah pengambilan *dataset*, yang bertujuan untuk memperoleh data visual yang representatif terhadap kondisi nyata di area kerja robot SCARA T3. Pada penelitian ini, data diperoleh dalam bentuk rekaman video. Video yang diambil dapat dilihat pada gambar 3.7.



Gambar 3. 7. Video dataset

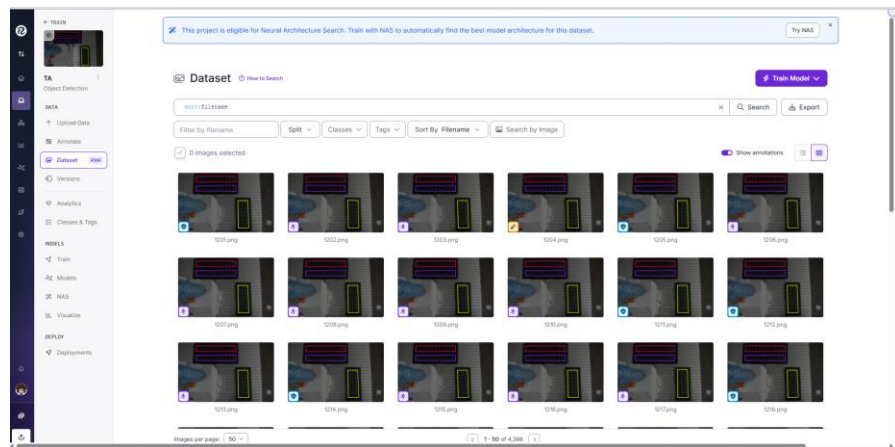
Video direkam dengan resolusi *Full HD*, untuk menangkap variasi posisi, orientasi, serta kondisi pencahayaan dari *pallet*. Terdapat tiga buah video yang masing masing memiliki *point of view* yang berbeda, supaya deteksi lebih akurat dengan total durasi video 2 menit 25,5 detik. Setelah proses perekaman selesai, video kemudian diubah menjadi serangkaian *frame* citra statis (*image sequence*). Dari hasil proses ekstraksi terhadap tiga video yang direkam, diperoleh total 4.336 gambar dikarenakan video memiliki frame 30 fps (frame per second) $145,53 \times 30 = 4.366$ gambar yang berhasil dihasilkan dari pemisahan setiap *frame* video. Seluruh gambar tersebut kemudian digunakan sebagai *dataset* utama dalam proses pelatihan model deteksi objek. Jumlah dan distribusi *dataset* ini dapat dilihat pada gambar 3.8.

Gambar 3. 8. Hasil gambar dari *frame* video

b. Labelling dataset

Setelah *dataset* dalam bentuk gambar berhasil diperoleh, tahap selanjutnya adalah pelabelan (*labelling*) menggunakan platform *Roboflow*.

Platform ini berfungsi untuk memberikan anotasi berupa *bounding box* pada objek-objek target yang ingin dideteksi oleh model, yaitu area *pallet*. Gambar sebanyak 4.366 *frame* di *upload* ke *roboflow* yang dapat dilihat pada gambar 3.9.



Gambar 3. 9. Gambar di *upload* ke *roboflow*

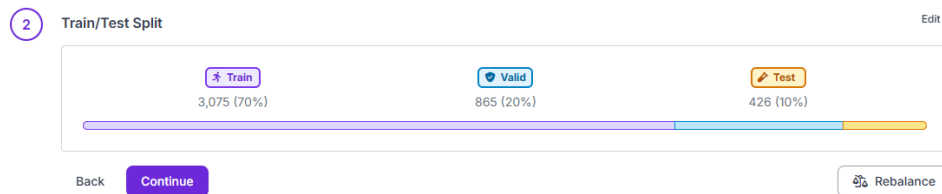
Kemudian dilakukan proses anotasi manual untuk menandai *pallet*. Setiap anotasi diberi label sesuai kelasnya “*pallet_pick*”, “*pallet_place1*”, “*pallet_place2*”. Agar model memiliki daya generalisasi tinggi, dilakukan pula proses augmentasi data seperti rotasi acak ($\pm 15^\circ$), perubahan kecerahan, *scaling*, dan *flipping*. Proses *labelling* pada roboflow dapat dilihat pada gambar 3.10.



Gambar 3. 10. Proses *labelling* gambar

Dataset kemudian dipartisi menjadi tiga subset, yaitu data latih (70%) sebanyak 3.075 gambar, data validasi (20%) sebanyak 865 gambar, dan data uji (10%) sebanyak 426 gambar. Pembagian ini bertujuan untuk melatih

model, melakukan *tuning* parameter, serta mengevaluasi performa model secara objektif. Pembagian data dapat dilihat pada gambar berikut:



Gambar 3. 11. Partisi *dataset*

c. *Training data*

Tahap terakhir adalah pelatihan model (*training data*) menggunakan Google Colab yang terintegrasi dengan *repository* resmi YOLO26. *File dataset* hasil ekspor dari *Roboflow* diunggah ke Google Drive, kemudian dihubungkan dengan Colab melalui *script* Python.

Sebelum pelatihan, dilakukan konfigurasi parameter pelatihan seperti $epoch = 100$, $batch\ size = 16$, $image\ size = 640 \times 640$, serta penggunaan *optimizer* MuSGD (*optimizer* baru khas YOLO26 yang menggabungkan SGD dan Muon) dengan *learning rate* 0.01.

Selama pelatihan, sistem menghitung nilai *loss function* yang mencakup *bounding box regression loss* (L_{box}), *objectness loss* (L_{obj}), dan *classification loss* (L_{cls}). Hasil terbaik dipilih berdasarkan nilai *mean Average Precision* tertinggi serta *validation loss* terendah. Model terbaik kemudian disimpan dalam *file* best.pt (format model YOLO26 dari Ultralytics). Proses *training data* menggunakan *Google Collab* dapat dilihat pada gambar 3.12.

$$\text{col} = \left\lfloor \frac{x_c - x_1}{\text{cell_width}} \right\rfloor + 1 \quad (12)$$

$$\text{row} = \left\lfloor \frac{y_c - y_1}{\text{cell_height}} \right\rfloor + 1 \quad (13)$$

Hasil dari proses ini adalah koordinat diskrit dalam bentuk pasangan (*col, row*) yang merepresentasikan posisi objek pada *pallet*.

Koordinat yang telah diperoleh kemudian digunakan sebagai referensi dalam perintah robot, seperti *Jump Pallet(1, col, row)* untuk proses pengambilan dan *Jump Pallet(2, col, row)* untuk proses penempatan. Dengan demikian, robot tidak perlu mengetahui koordinat absolut dalam satuan milimeter, melainkan cukup menggunakan indeks *grid* yang telah dipetakan sebelumnya.

Tabel 3. 1. Tampilan *Grid*

(1.8)		
		(3.5)
	(2.2)	
		(3.1)

Koordinat diatas lah yang dikirimkan ke robot agar robot bergerak ke objek *workpiece* yang terdeteksi. Urutan pengambilan dimulai dari koordinat X, Y terkecil berurutan hingga X,Y terbesar.

6. Perancangan *Software*

Dalam perancangan *software*, beberapa perangkat lunak pendukung dibutuhkan untuk membangun sistem, perangkat lunak tersebut antara lain:

a. *Graphical User Interface*

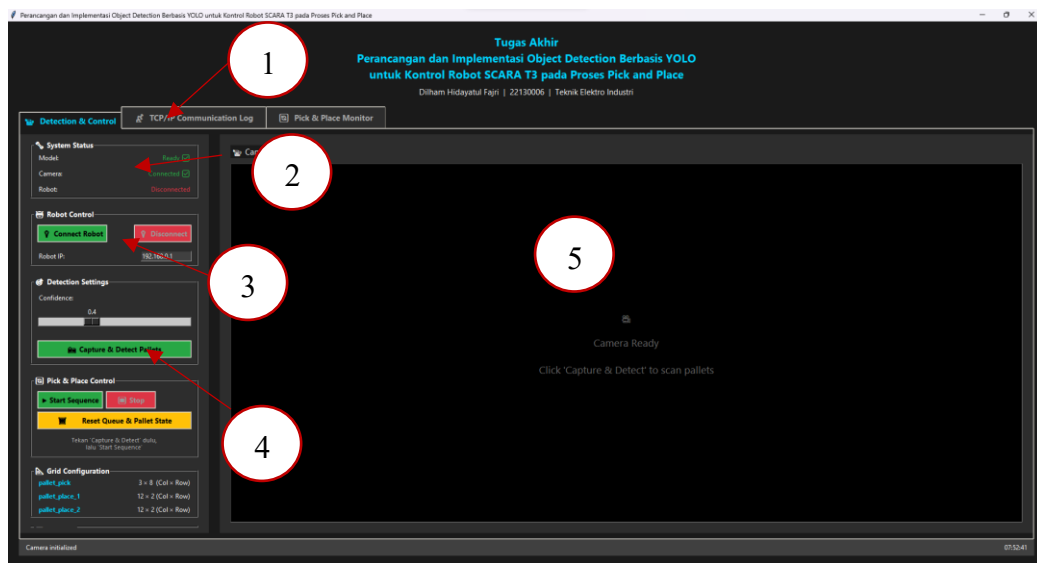
GUI merupakan antarmuka yang dirancang untuk mengintegrasikan sistem deteksi objek berbasis YOLO dengan robot SCARA T3 dalam

implementasi proses *pick and place*. Antarmuka ini dibagi menjadi dua bagian utama, yaitu panel kontrol di sisi kiri dan tampilan kamera di sisi kanan. Pada panel kontrol, tersedia menu *System status* untuk menampilkan kondisi terkini koneksi kamera dan robot sehingga pengguna dapat memastikan perangkat siap digunakan. Bagian *Robot Control* dilengkapi tombol *Connect Robot* yang memungkinkan pengguna melakukan inisialisasi komunikasi antara perangkat lunak dengan robot, serta mengatur perintah dasar pergerakan.

Selanjutnya, pada bagian *Detection Settings*, pengguna dapat memilih model deteksi YOLO yang digunakan serta memproses citra melalui tombol *Capture & Process Image*, di mana hasil deteksi objek akan diekstraksi dalam bentuk *bounding box* dan koordinat. Data koordinat ini secara otomatis diterjemahkan menjadi referensi posisi robot untuk menentukan titik pengambilan (*pick point*). Di sisi lain, menu *Statistics* menampilkan informasi analitis berupa jumlah objek yang berhasil terdeteksi, waktu komputasi, serta opsi *Reset Stats* untuk mengulang penghitungan.

Pada sisi kanan GUI, jendela *Camera Feed* menyajikan tampilan visual dari kamera, lengkap dengan anotasi hasil deteksi berupa *bounding box* dan label objek. Keseluruhan integrasi ini tidak hanya memberikan kemudahan dalam memantau performa sistem, tetapi juga meningkatkan akurasi dan efisiensi proses *pick and place* karena operator dapat langsung memvalidasi hasil deteksi serta memastikan koordinat objek sesuai dengan pergerakan robot.

Penjelasan di atas dapat dilihat pada Gambar 3.13, yang menampilkan rancangan *Graphical User Interface* (GUI). Antarmuka ini dirancang untuk menjadi jembatan interaktif antara sistem deteksi objek berbasis YOLO dengan robot SCARA T3 pada implementasi proses *pick and place*.



Gambar 3. 13. Tampilan GUI

- 1) Tab Deteksi dan Kontrol Menu utama sistem yang menjadi pusat kendali seluruh proses operasi. Melalui tab ini pengguna dapat mengakses dan menjalankan semua fungsi sistem berbasis vision dan pengendalian robot secara terintegrasi dalam satu tampilan.
- 2) Status Sistem Menampilkan kondisi seluruh perangkat secara *real-time* yang mencakup tiga indikator utama yaitu status koneksi kamera, status model deteksi YOLO26, dan status koneksi robot. Fitur ini memungkinkan pengguna memantau kesiapan sistem sebelum dan selama operasi berlangsung.
- 3) Kontrol Koneksi Robot Menyediakan fungsi koneksi dan pemutusan komunikasi antara sistem visi dan robot SCARA T3 melalui protokol TCP/IP. Melalui fitur ini pengguna dapat memastikan robot dalam kondisi siap menerima perintah gerak sebelum proses *pick and place* dijalankan.
- 4) Pengaturan Deteksi Berfungsi untuk mengatur parameter deteksi model YOLO26, khususnya nilai *confidence threshold* yang menentukan tingkat keyakinan minimum model dalam mengenali objek. Pada bagian ini juga tersedia tombol "*Capture & Process*" yang digunakan untuk

mengambil citra dari kamera dan menjalankan proses deteksi objek secara langsung dalam satu perintah.

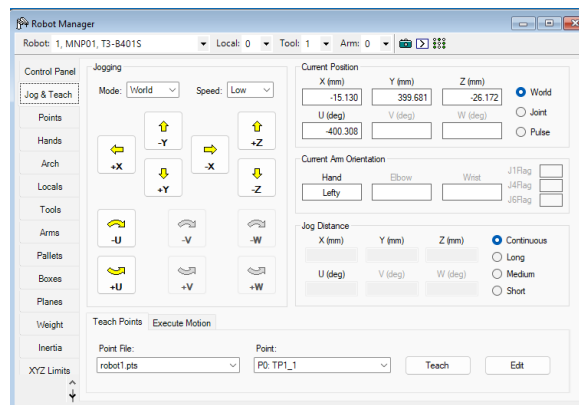
- 5) Area Tampilan Hasil Deteksi Menampilkan *output* visual hasil proses deteksi secara *real-time* di sisi kanan antarmuka. Area ini memvisualisasikan hasil identifikasi *pallet*, pemetaan *grid* 3×8, dan posisi *workpiece* yang terdeteksi, dilengkapi dengan anotasi berupa *bounding box* dan koordinat *grid* untuk mempermudah interpretasi hasil deteksi oleh pengguna.

b. Epson RC+

Aplikasi Epson RC+ merupakan perangkat lunak resmi yang dikembangkan oleh Epson untuk memprogram, mengontrol, serta melakukan simulasi terhadap robot industri Epson, termasuk robot SCARA T3. Aplikasi ini berfungsi sebagai *integrated development environment* (IDE) yang menyediakan berbagai fitur untuk mendukung proses perancangan, pengujian, hingga implementasi sistem robotik dalam suatu proses industri.

a. *Jog and Teach Robot*

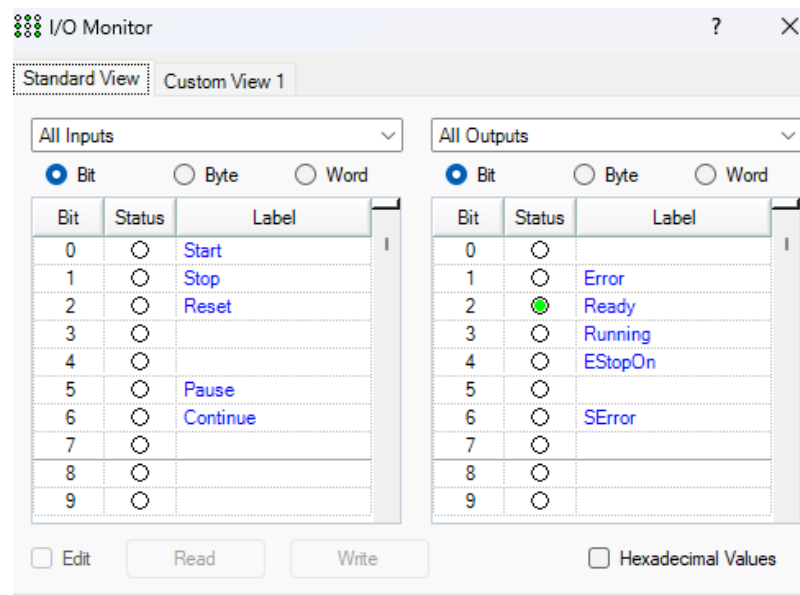
Jog and teach digunakan untuk menggerakkan lengan robot atau jog yang terhubung secara virtual maupun terhubung dengan *ethernet* keposisi yang telah ditentukan untuk sistem otomasinya. kemudian operator melakukan *teach* untuk menyimpan posisi robot atau *point teaching*. Tampilan *jog and teach* dapat dilihat pada gambar 3.14.



Gambar 3. 14. Tampilan *Jog and Teach*

b. *I/O Monitor*

I/O Monitor pada aplikasi Epson RC+, yang berfungsi untuk memantau serta mengendalikan sinyal *Input* dan *Output* digital pada robot Epson SCARA T3. Fitur ini sangat penting dalam proses integrasi robot dengan perangkat eksternal maupun dalam *debugging* program, karena memungkinkan pengguna untuk melihat status sinyal. Tampilan *I/O Monitor* dapat dilihat pada gambar 3.15.

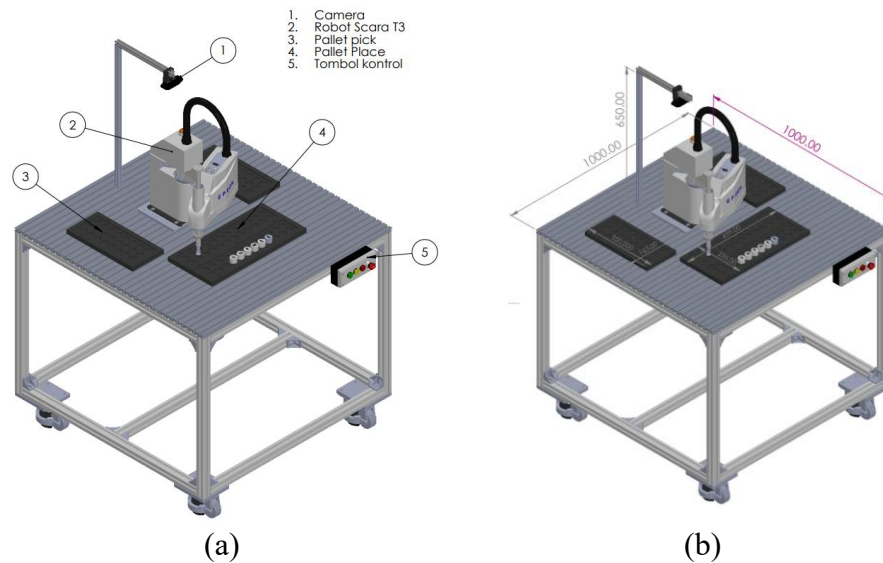


Gambar 3. 15. Tampilan *I/O Monitor*

7. Perancangan *Hardware*

a. Desain sistem

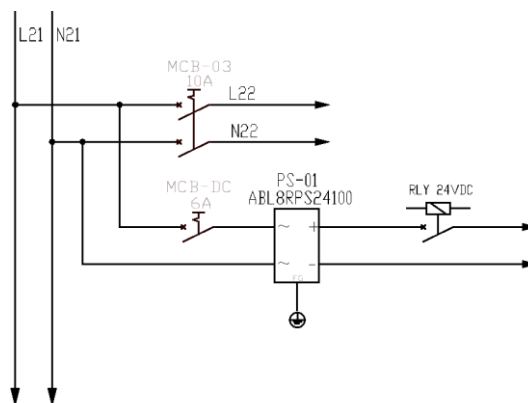
Perancangan *hardware* pada sistem *pick and place* ini terdiri dari beberapa komponen utama. Robot yang digunakan adalah Epson SCARA T3-B401, dipasang pada meja kerja berbahan profil. Pada sisi atas meja, dipasang kamera Logitech C920 yang terhubung ke komputer, ditempatkan pada tiang penyangga dengan ketinggian 65 cm untuk memperoleh bidang pandang yang optimal terhadap area *pallet*.



Gambar 3. 16. Desain Sistem: a. Nama *part*. b. Dimensi

b. Perancangan *Electrical*

Perancangan *elektrikal* bertujuan merancang rangkaian yang diterapkan pada alat serta proteksi. Pada gambar 3.17 menampilkan gambaran rangkaian *power* untuk *controller robot*, *power supply unit* DC. Sumber tegangan AC masuk ke main MCB, lalu keluaran main MCB ke robot, Kemudian *input* AC diparalel dengan MCB 6A untuk *Power Supply* DC.

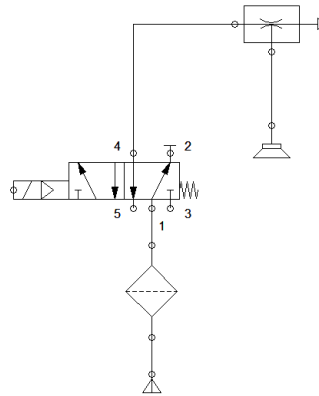


Gambar 3. 17. Rangkaian *Power*

c. Perancangan *Pneumatik*

Perancangan sistem *pneumatik* bertujuan untuk memberikan visualisasi alur rangkaian *pneumatik* dari setiap komponen yang digunakan dalam proses otomasi. Visualisasi ini diperlukan untuk memastikan bahwa sistem

dapat berfungsi dengan baik serta mempermudah dalam proses perakitan dan penanganan gangguan apabila terjadi kerusakan atau kesalahan dalam pengoperasian. Rancangan *pneumatik* dapat dilihat pada gambar 3.18.



Gambar 3. 18. Rangkaian *Vacuum Suction Cup*

Sistem ini dilengkapi dengan unit *pneumatik* yang terdiri dari kompresor, *Air Regulator*, solenoid valve 24VDC, *vacuum generator* (*venturi*), serta *suction cup* yang dipasang pada *end effector* robot. Udara bertekanan dari kompresor dialirkan melalui *Air Regulator* menuju *vacuum generator*, yang kemudian diatur oleh solenoid valve untuk mengaktifkan atau mematikan hisapan pada *suction cup*. Seluruh komponen *pneumatik* ditempatkan di bagian bawah meja agar lebih aman dan rapi, sedangkan *tubing* fleksibel disalurkan melalui *cable carrier* menuju robot.

C. Implementasi Sistem

Penelitian dan implementasi sistem *object detection* berbasis YOLO26 untuk kontrol robot SCARA T3 pada proses *pick and place* dilakukan di lingkungan *Teaching Factory* (TEFA). Lingkungan *Teaching Factory* dipilih karena merepresentasikan kondisi industri nyata dalam skala pembelajaran, baik dari sisi penggunaan peralatan robot industri, alur kerja produksi, maupun penerapan sistem otomasi.

Implementasi sistem dilakukan dengan memanfaatkan fasilitas robot SCARA yang tersedia di *Teaching Factory*, sehingga proses perancangan, integrasi, dan pengujian sistem dapat dilakukan secara langsung pada sistem yang menyerupai kondisi operasional industri. Melalui implementasi di

Teaching Factory, sistem yang dikembangkan tidak hanya bersifat konseptual, tetapi juga aplikatif dan dapat digunakan sebagai media pembelajaran serta simulasi proses produksi berbasis robotika industri.

D. Pengujian

Tahap pengujian dilakukan untuk mengetahui kinerja sistem *object detection* berbasis YOLO26 dan integrasinya dengan robot SCARA T3 pada proses *pick and place*. Pengujian difokuskan pada empat parameter utama, yaitu akurasi deteksi objek, waktu *latency* komunikasi data, dan *cycle time* robot. Data pengujian diperoleh melalui pengamatan langsung dan pencatatan sistem selama proses pengujian berlangsung.

1. Evaluasi model YOLO26

Pengujian akurasi deteksi objek dilakukan untuk mengevaluasi kinerja model YOLO26 dalam mendeteksi workpiece yang berada pada area kerja robot SCARA T3. Pengujian dilakukan menggunakan metode validasi (*validation*) yang disediakan oleh *framework* YOLO, yaitu dengan menjalankan proses evaluasi model terhadap *dataset* pengujian yang telah dilengkapi dengan anotasi *ground truth*. *Dataset* pengujian memuat variasi posisi dan orientasi *workpiece* di dalam area kerja robot.

Metode evaluasi ini diadopsi dari penelitian-penelitian sebelumnya yang menggunakan pendekatan serupa dalam menilai performa model YOLO pada sistem deteksi objek berbasis *deep learning*. Penelitian terdahulu menyatakan bahwa penggunaan metode validasi bawaan YOLO dengan *metrik Intersection over Union (IoU)*, *Precision*, *Recall* merupakan pendekatan yang objektif dan terstandar untuk mengevaluasi kinerja deteksi objek secara menyeluruh. Oleh karena itu, metode ini dipilih karena mampu memberikan hasil evaluasi yang sssskonsisten, terukur, dan dapat dibandingkan dengan penelitian sejenis.

2. *Latency* Komunikasi Data (TCP/IP)

Waktu *latency* dinyatakan dalam satuan milidetik (ms) yang menunjukkan durasi pengiriman data koordinat dari PC ke robot SCARA T3 melalui jaringan TCP/IP. Data *latency* diperoleh dari selisih waktu antar pengiriman paket data

secara berurutan yang dikirimkan oleh sistem visi berbasis Python ke kontroler robot.

$$\Delta t = t_n - t_{n-1} \quad (14)$$

Di mana:

Δt = nilai *latency* antar paket data (milidetik)

t_n = *timestamp* pencatatan waktu pada paket data ke-n

t_{n-1} = *timestamp* pencatatan waktu pada paket data ke-(n-1)

Proses pengambilan waktu komunikasi dilakukan menggunakan pencatatan *timestamp* pada program Python. Waktu t_{n-1} dicatat pada saat paket data koordinat hasil deteksi objek ke-(n-1) dikirimkan dari PC ke robot SCARA melalui komunikasi TCP/IP. Waktu t_n dicatat pada saat paket data koordinat berikutnya ke-n dikirimkan. Selisih antara t_n dan t_{n-1} digunakan sebagai nilai *latency* komunikasi antar paket data, sehingga diperoleh gambaran interval waktu aktual yang terjadi antara setiap pengiriman perintah gerak robot secara berurutan.

Pengujian dilakukan sebanyak 5 percobaan dengan masing-masing percobaan terdiri dari 16 data pengiriman paket, sehingga setiap percobaan menghasilkan 15 nilai *latency* yang dihitung dari selisih antar *timestamp* berurutan. Nilai *latency* rata-rata, minimum, maksimum, dan standar deviasi dari seluruh percobaan kemudian dianalisis dan dibandingkan dengan standar *latency* komunikasi industri untuk mengevaluasi kelayakan sistem komunikasi TCP/IP yang diimplementasikan.

3. Cycle Time

Cycle time robot dinyatakan dalam satuan waktu (detik) yang menunjukkan durasi satu siklus kerja *pick and place*. Satu siklus kerja dimulai dari robot bergerak menuju *workpiece*, melakukan proses *pick*, memindahkan semua *workpiece* yang terdeteksi ke pallet tujuan, hingga robot kembali ke posisi awal atau posisi siap. Nilai *cycle time* diperoleh dengan menggunakan program python dengan menggunakan fungsi *time* yang berjalan saat deteksi dimulai dan berakhir ketika robot kembali ke *home*.

4. Penerapan deteksi *workpiece* dan *pallet*

Pengujian penerapan deteksi *workpiece* dan *pallet* dilakukan untuk mengevaluasi kemampuan sistem berbasis YOLO26 dalam mendeteksi dan menghitung jumlah *workpiece* secara akurat pada kondisi operasional nyata. Pengujian ini bertujuan untuk mengetahui tingkat akurasi sistem dalam mengenali objek serta konsistensi hasil deteksi pada dua kondisi *pallet* yang berbeda, yaitu kondisi *pallet pick* dan kondisi *pallet place* 1.

Skenario pengujian dirancang dengan memvariasikan jumlah *workpiece* yang ditempatkan pada *pallet* dari 1 hingga 10 buah. Setiap variasi jumlah *workpiece* diuji sebanyak 5 kali percobaan untuk memastikan konsistensi dan keandalan sistem dalam kondisi yang berulang. Dengan demikian, total pengujian yang dilakukan adalah sebanyak 50 percobaan untuk masing-masing kondisi *pallet*, yang terdiri dari 10 variasi jumlah *workpiece* dikalikan 5 kali pengulangan.

Proses deteksi dilakukan dengan mengambil citra dari kamera menggunakan tombol *Capture & Process* pada antarmuka sistem, kemudian model YOLO26 menjalankan inferensi untuk mendeteksi posisi *pallet* dan menghitung jumlah *workpiece* yang teridentifikasi dalam area *grid pallet*. Hasil deteksi berupa jumlah *workpiece* yang terdeteksi oleh sistem kemudian dibandingkan dengan jumlah *workpiece* aktual yang ditempatkan secara manual pada *pallet* untuk menghitung tingkat akurasi deteksi.

E. Evaluasi

Tahap evaluasi dilakukan untuk menilai kinerja sistem *object detection* berbasis YOLO26 yang terintegrasi dengan robot SCARA T3 pada proses *pick and place*. Evaluasi sistem dilakukan berdasarkan hasil pengujian yang telah dilakukan, meliputi akurasi deteksi objek, waktu *latency* komunikasi data, *pick success rate*, dan *cycle time* robot. Parameter-parameter tersebut digunakan sebagai indikator untuk mengetahui tingkat keberhasilan sistem dalam memenuhi tujuan penelitian.

Evaluasi akurasi deteksi objek dilakukan dengan menganalisis nilai persentase deteksi benar terhadap total percobaan. Nilai akurasi ini digunakan

G. Rancangan Anggaran Biaya

Tabel 3. 3. Rancangan Anggaran Biaya

No	Bahan	Jumlah	Satuan	Harga	Total
1	Aluminium Profile 20 x 20	100	Cm	Rp 1.200,00	Rp 120.000,00
2	Cap Screw m6	10	Pcs	Rp 1.500,00	Rp 15.000,00
3	T nut m6	10	Pcs	Rp 1.000,00	Rp 10.000,00
4	Aluminium Bracket Corner 20 x 20	4	Buah	Rp 5.500,00	Rp 22.000,00
5	Camera Webcam C920	1	Item	Rp 1.000.000,00	Rp 1.000.000,00
6	Converter USB to LAN	1	Item	Rp 120.000,00	Rp 120.000,00
7	Flat End mill 6mm	1	Item	Rp 50.000,00	Rp 50.000,00
8	Kabel LAN	5	Meter	Rp 25.000,00	Rp 125.000,00
Grand Total					Rp 1.462.000,00

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil implementasi serta analisis dari sistem yang telah dirancang dan dijelaskan pada Bab III. Pengujian dilakukan untuk mengevaluasi kinerja model deteksi objek berbasis YOLO, akurasi transformasi koordinat, serta performa sistem secara keseluruhan dalam mengendalikan robot pada proses pick-and-place.

Proses evaluasi dilakukan secara bertahap sesuai dengan alur metodologi, dimulai dari analisis hasil pelatihan model, pengujian kinerja deteksi menggunakan metrik evaluasi, hingga implementasi sistem pada kondisi *real-time*. Selain itu, dilakukan pula pengujian terhadap parameter penting seperti *latency*, *cycle time*.

Hasil yang diperoleh kemudian dianalisis untuk mengetahui tingkat akurasi, kecepatan respon, serta keandalan sistem dalam berbagai kondisi pengujian. Analisis ini juga bertujuan untuk mengidentifikasi faktor-faktor yang mempengaruhi performa sistem, baik dari sisi algoritma, kualitas data, maupun integrasi perangkat keras. Dengan demikian, Bab ini tidak hanya menyajikan hasil pengujian, tetapi juga memberikan pembahasan yang komprehensif terkait keterkaitan antara metode yang digunakan dengan performa sistem yang dihasilkan.

A. Analisis kinerja Model YOLOv26

Analisis kinerja model dilakukan menggunakan beberapa metrik evaluasi, yaitu *precision*, *recall*, serta ditinjau dari konvergensi proses pelatihan melalui grafik loss. Selain itu, dilakukan analisis terhadap kurva evaluasi seperti *precision–confidence*, *recall–confidence*, dan *F1-score* untuk menentukan threshold optimal yang menghasilkan keseimbangan terbaik antara ketepatan dan kelengkapan deteksi.

Hasil evaluasi ini digunakan untuk menilai sejauh mana model mampu menggeneralisasi data serta mengidentifikasi potensi kesalahan deteksi, seperti *false positive* dan *false negative*.

Dengan demikian, analisis ini menjadi dasar dalam menentukan kelayakan model untuk diimplementasikan pada sistem *real-time* yang terintegrasi dengan robot.

1. Konvergensi *loss model*

Konvergensi *loss model* merupakan salah satu indikator utama dalam mengevaluasi keberhasilan proses pelatihan model YOLOv26. Analisis ini bertujuan untuk mengetahui sejauh mana model mampu mempelajari pola dari data pelatihan melalui penurunan nilai *loss* pada setiap *epoch*.

Pada penelitian ini, parameter *loss* yang diamati meliputi *box loss*, *classification loss*, dan *distribution focal loss (DFL)*. Ketiga komponen tersebut merepresentasikan kesalahan model dalam memprediksi lokasi objek, kelas objek, serta kualitas distribusi *bounding box*. Selama proses pelatihan, nilai *loss* diharapkan mengalami penurunan secara bertahap hingga mencapai kondisi stabil, yang menunjukkan bahwa model telah mencapai konvergensi.

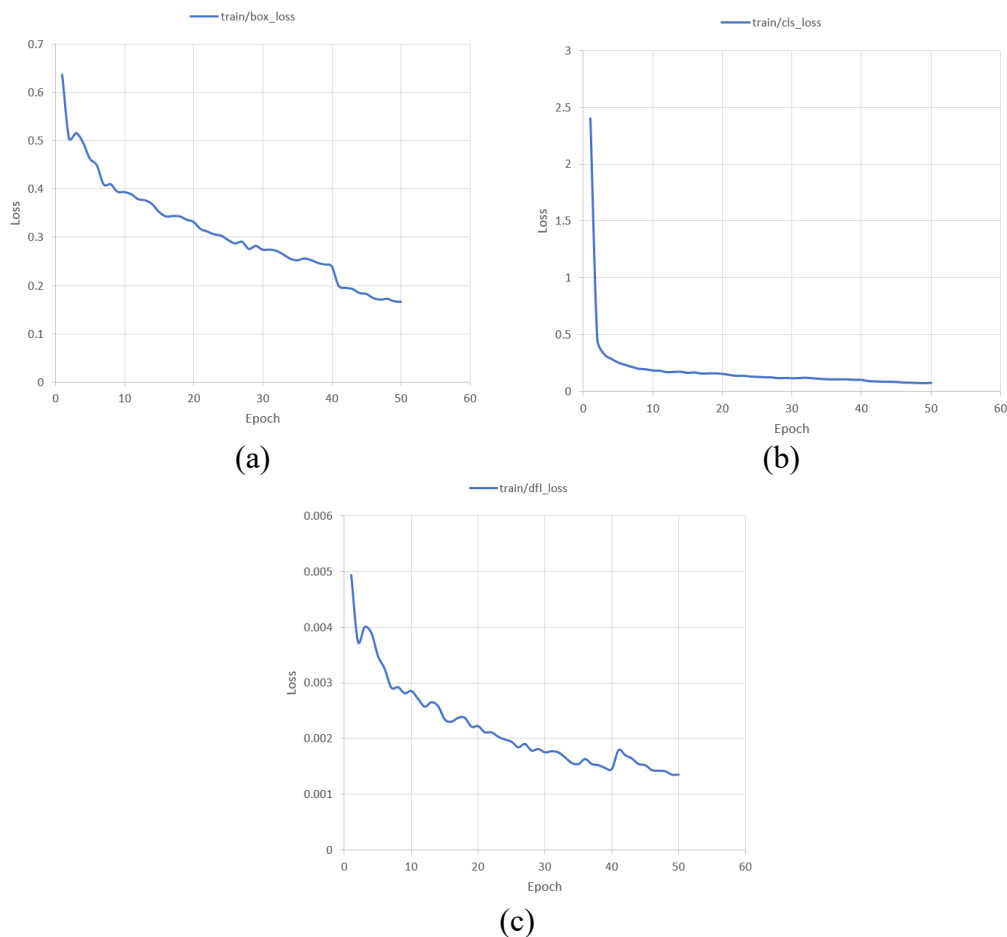
Selain itu, perbandingan antara *loss* pada data pelatihan dan data validasi juga dianalisis untuk mengidentifikasi kemungkinan terjadinya *overfitting* atau *underfitting*. Dengan demikian, analisis konvergensi *loss* tidak hanya menunjukkan performa pembelajaran model, tetapi juga memberikan gambaran mengenai kemampuan generalisasi model terhadap data yang belum pernah dilihat sebelumnya. Pada tabel berikut disajikan hasil nilai *loss model* yang diperoleh selama proses pelatihan. Nilai *loss* tersebut digunakan untuk mengevaluasi kinerja model dalam mempelajari data, yang meliputi *box loss*, *classification loss*, dan *distribution focal loss (DFL)* pada setiap *epoch*.

Tabel 4. 1. Hasil konvergensi *loss model*

<i>epoch</i>	<i>train/box loss</i>	<i>train/cls loss</i>	<i>train/dfn loss</i>
1	0.63642	2.39849	0.00493
10	0.39338	0.1831	0.00285
20	0.33138	0.15465	0.00222
30	0.27354	0.11529	0.00175
40	0.23902	0.10149	0.00145
50	0.16549	0.07482	0.00135

Pada epoch 1, *loss box* pelatihan, *loss* klasifikasi, dan *loss* DFL masing-masing adalah 0,6364, 2,3985, dan 0,00493, yang mencerminkan ketidakpastian awal model yang tinggi. Pada epoch 50, nilai-nilai ini telah konvergen menjadi 0,1655, 0,0748, dan 0,00135. Penurunan yang konsisten pada ketiga jenis *loss* menunjukkan bahwa proses pembelajaran berjalan dengan baik dan stabil. Hal ini menandakan bahwa model berhasil meminimalkan kesalahan prediksi baik dari sisi lokasi objek maupun klasifikasinya.

Untuk memperjelas analisis terhadap proses pembelajaran model, data *loss* pada tabel selanjutnya divisualisasikan dalam bentuk grafik



Gambar 4. 1. Grafik *loss* (a) *box_loss*; (b) *cls_loss*; (c) *dfl_loss*

Kurva yang cenderung stabil pada akhir *epoch* mengindikasikan bahwa model telah mencapai kondisi konvergensi, di mana penurunan *loss* tidak lagi signifikan. Hal ini menunjukkan bahwa model telah mencapai performa

optimal dalam mempelajari *dataset* yang digunakan. Selain itu, tidak terdapat fluktuasi yang signifikan pada grafik, yang menandakan bahwa proses pelatihan berlangsung secara stabil tanpa adanya indikasi ketidakstabilan atau divergensi.

2. *Precision, Recall, F-1 Score dan Confusion Matrix*

Performa model dievaluasi pada set data uji berisi 426 gambar yang ditahan menggunakan metrik deteksi objek standar: *Presisi* (P), *Recall* (R), F1-Score, dan rata-rata Presisi Metrik-metrik ini secara formal didefinisikan sebagai berikut:

$$Precision = \frac{TP}{TP + FP} \quad (15)$$

$$Recall = \frac{TP}{TP + FN} \quad (16)$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (17)$$

Evaluasi kinerja model YOLOv26 dilakukan menggunakan metrik *precision*, *recall*, *F1-score*, serta *confusion matrix* untuk mengukur tingkat akurasi deteksi objek secara komprehensif. Metrik-metrik ini digunakan untuk menilai kemampuan model dalam mengklasifikasikan objek dengan benar serta mengidentifikasi kesalahan yang terjadi selama proses deteksi.

Precision digunakan untuk mengukur tingkat ketepatan model dalam memprediksi objek, yaitu perbandingan antara jumlah prediksi benar (*true positive*) terhadap seluruh prediksi positif yang dihasilkan oleh model. Nilai *precision* yang tinggi menunjukkan bahwa model memiliki tingkat kesalahan *false positive* yang rendah. Sementara itu, *recall* digunakan untuk mengukur kemampuan model dalam mendeteksi seluruh objek yang ada, yaitu perbandingan antara jumlah *true positive* terhadap total objek yang seharusnya terdeteksi. Nilai *recall* yang tinggi menunjukkan bahwa model mampu meminimalkan *false negative*.

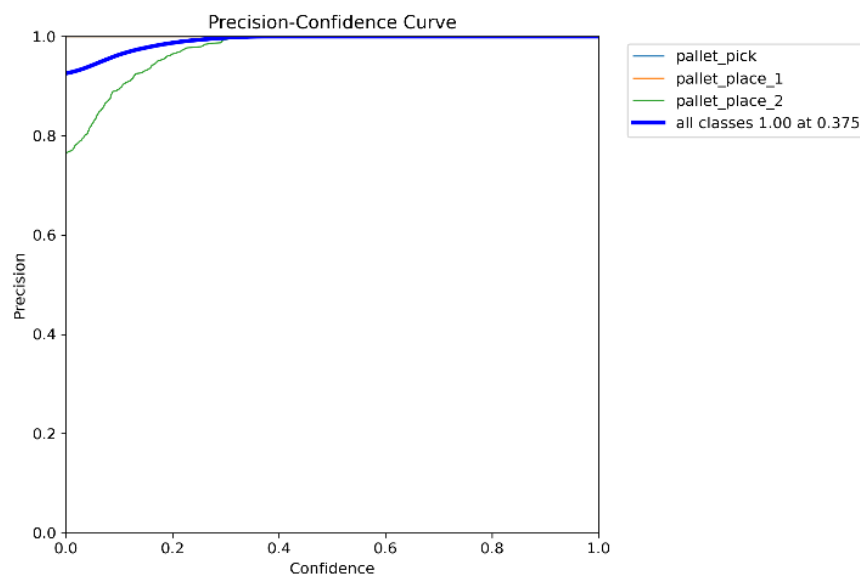
Untuk memperoleh keseimbangan antara *precision* dan *recall*, digunakan metrik *F1-score* yang merupakan rata-rata harmonik dari kedua nilai tersebut. Nilai *F1-score* yang tinggi menunjukkan bahwa model

memiliki performa yang seimbang antara ketepatan dan kelengkapan deteksi.

Selain itu, *confusion matrix* digunakan untuk memberikan gambaran yang lebih rinci mengenai performa model dalam mengklasifikasikan setiap kelas objek. Matriks ini menunjukkan jumlah prediksi benar dan salah pada masing-masing kelas, sehingga dapat diketahui distribusi *true positive*, *false positive*, *false negative*, serta *true negative*.

a. *Precision Curve*

Untuk mengevaluasi tingkat ketepatan model dalam menghasilkan prediksi, digunakan kurva *precision terhadap confidence*. Kurva ini menggambarkan hubungan antara nilai *confidence threshold* dengan tingkat *precision* yang dihasilkan oleh model pada setiap kelas objek. Gambar berikut menunjukkan hubungan antara nilai *precision* terhadap *confidence threshold*.



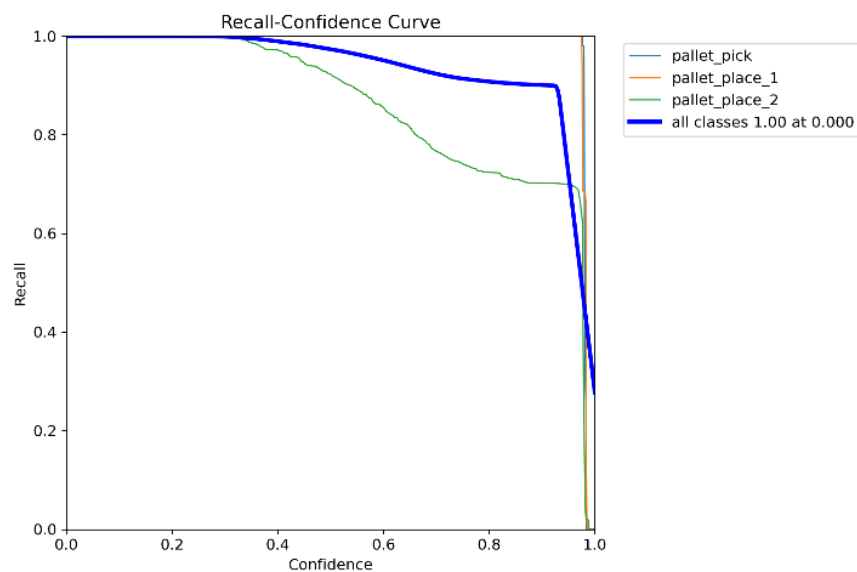
Gambar 4. 2. Kurva Presisi terhadap *confidence*

Berdasarkan grafik *precision-confidence*, terlihat bahwa nilai *precision* meningkat seiring dengan kenaikan nilai *confidence threshold*. Pada *threshold* tertentu, *precision* mencapai nilai maksimum mendekati 1.00, yang menunjukkan bahwa model memiliki tingkat kesalahan *false positive* yang sangat rendah.

Nilai *precision* terbaik diperoleh pada *confidence* sekitar 0.375, di mana model mampu menghasilkan prediksi yang sangat akurat untuk seluruh kelas. Hal ini menunjukkan bahwa model memiliki kemampuan yang sangat baik dalam membedakan objek dengan latar belakang maupun antar kelas objek.

b. *Recall Curve*

Selanjutnya, untuk menganalisis kemampuan model dalam mendeteksi seluruh objek yang ada, digunakan kurva *recall* terhadap *confidence*. Kurva ini menunjukkan hubungan antara nilai *confidence threshold* dengan tingkat keberhasilan model dalam mendeteksi objek (*true positive*). Gambar berikut menunjukkan hubungan antara nilai *recall* terhadap *confidence threshold*.



Gambar 4. 3. Kurva *Recall* terhadap *confidence*

Grafik *recall-confidence* menunjukkan bahwa nilai *recall* cenderung tinggi pada *confidence* rendah dan mulai menurun seiring meningkatnya *threshold*. Hal ini terjadi karena model menjadi lebih selektif dalam menerima prediksi pada nilai *confidence* yang lebih tinggi.

Pada *confidence* mendekati 0, *recall* mencapai nilai maksimum sebesar 1.00, yang berarti seluruh objek berhasil terdeteksi. Namun, ketika *confidence* mendekati 1, *recall* mengalami penurunan yang cukup

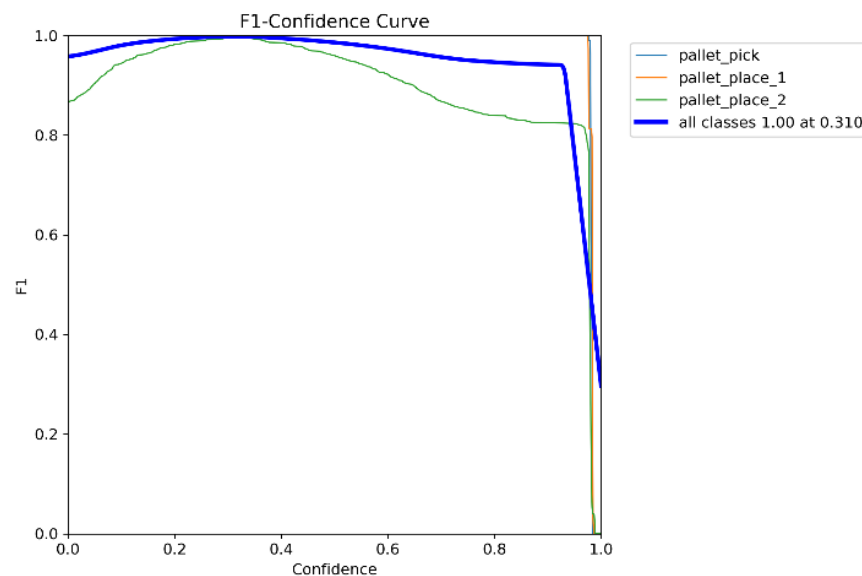
signifikan, yang menunjukkan adanya objek yang tidak terdeteksi (*false negative*).

c. *F-1 Curve*

Untuk memperoleh keseimbangan antara *precision* dan *recall*, digunakan kurva *F1-score terhadap confidence*. Kurva ini merupakan representasi performa gabungan yang mempertimbangkan ketepatan dan kelengkapan deteksi secara bersamaan.

Melalui kurva ini, dapat ditentukan nilai *confidence* optimal yang menghasilkan nilai *F1-score* tertinggi, sehingga memberikan performa terbaik dalam implementasi sistem. Gambar berikut menunjukkan hubungan antara nilai *F1-score* terhadap *confidence threshold*.

Grafik *F1-confidence* menunjukkan keseimbangan antara *precision* dan *recall*.



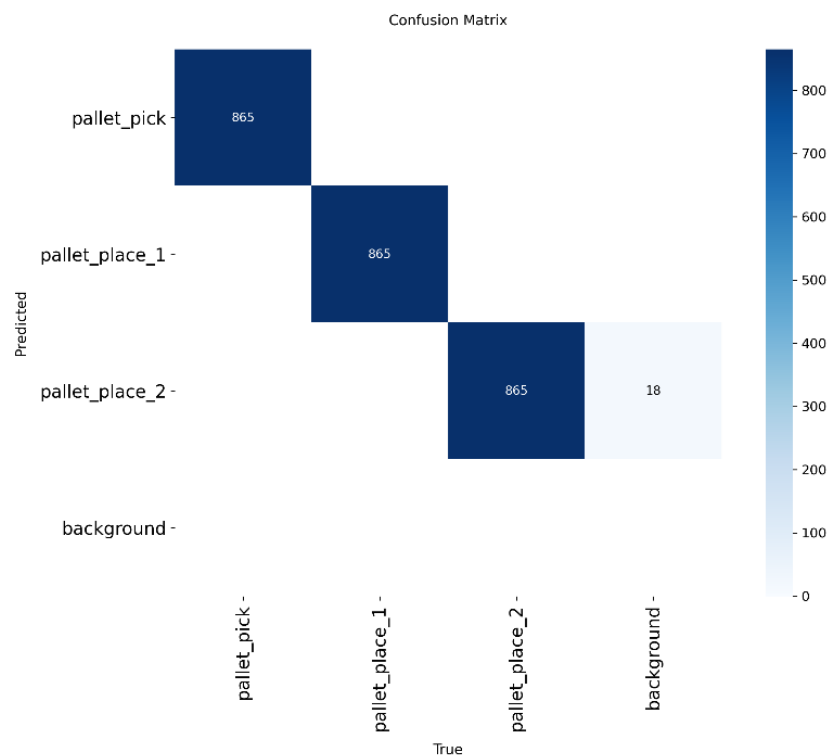
Gambar 4. 4. Kurva F1 terhadap *confidence*

Berdasarkan grafik, nilai *F1-score* mencapai nilai maksimum sebesar 1.00 pada *confidence* sekitar 0.310. Hal ini menunjukkan bahwa pada threshold tersebut, model memiliki keseimbangan terbaik antara kemampuan mendeteksi objek (*recall*) dan ketepatan prediksi (*precision*). Oleh karena itu, nilai *confidence* sekitar 0.31 dapat dianggap sebagai *threshold* optimal untuk implementasi sistem.

Selain itu, kurva F1 menunjukkan tren stabil pada rentang *confidence* menengah, yang mengindikasikan bahwa model memiliki performa yang konsisten dalam kondisi tersebut.

d. *Confusion Matrix*

Confusion matrix menunjukkan distribusi prediksi benar dan salah, sehingga dapat digunakan untuk mengidentifikasi pola kesalahan serta mengevaluasi performa model pada masing-masing kelas secara spesifik. Gambar berikut menunjukkan *confusion matrix* hasil klasifikasi model terhadap data pengujian.



Gambar 4. 5. *Confusion Matrix*

Berdasarkan *confusion matrix* yang ditampilkan, terlihat bahwa sebagian besar prediksi model berada pada diagonal utama, yang menunjukkan bahwa model mampu mengklasifikasikan objek dengan benar.

- 1) *pallet_pick*: seluruh data terklasifikasi dengan benar sebanyak 865 data

- 2) *pallet_place_1*: seluruh data terklasifikasi dengan benar sebanyak 865 data
- 3) *pallet_place_2*: sebagian besar data terklasifikasi dengan benar (865), namun terdapat 18 kesalahan yang diklasifikasikan sebagai kelas lain
- 4) *background*: hampir tidak terdapat kesalahan signifikan

B. Pengujian dan analisis *latency* komunikasi Data TCP/IP

Pengujian *latency* komunikasi data TCP/IP dilakukan untuk mengukur selisih waktu antar pengiriman paket data dari sistem visi berbasis Python ke kontroler robot Epson SCARA T3. *Latency* dihitung dengan mengurangi timestamp paket ke- n terhadap paket ke- $(n-1)$, sehingga diperoleh interval waktu aktual antar pengiriman perintah gerak robot secara berurutan. Pengujian dilakukan sebanyak 5 kali percobaan dengan masing masing 10 data pada kondisi jaringan lokal (LAN) dengan koneksi langsung antara PC *host* dan kontroler robot.

Tabel 4. 2. Data *Latency* percobaan ke-1

No	Data ke-n	Waktu Terkirim (h:m:s.ms)	<i>Latency</i> (ms) $\Delta t = t_n - t_{n-1}$
1	Data ke 1	17:14:04.124	-
2	Data ke 2	17:14:04.159	35
3	Data ke 3	17:14:04.172	13
4	Data ke 4	17:14:04.186	14
5	Data ke 5	17:14:04.218	32
6	Data ke 6	17:14:04.232	14
7	Data ke 7	17:14:04.249	17
8	Data ke 8	17:14:04.261	12
9	Data ke 9	17:14:04.293	32
10	Data ke 10	17:14:04.303	10
11	Data ke 11	17:14:04.317	14
12	Data ke 12	17:14:04.331	14
13	Data ke 13	17:14:04.360	29
14	Data ke 14	17:14:04.372	12
15	Data ke 15	17:14:04.385	13
16	Data ke 16	17:14:04.398	13
Rata-rata			18.27 ms

Tabel 4. 3. Data *Latency* percobaan ke-2

No	Data ke-n	Waktu Terkirim (h:m:s.ms)	<i>Latency</i> (ms) $\Delta t = t_n - t_{n-1}$
1	Data ke 1	17:16:52.206	-
2	Data ke 2	17:16:52.230	24
3	Data ke 3	17:16:52.249	19
4	Data ke 4	17:16:52.264	15
5	Data ke 5	17:16:52.297	33
6	Data ke 6	17:16:52.314	17
7	Data ke 7	17:16:52.328	14
8	Data ke 8	17:16:52.343	15
9	Data ke 9	17:16:52.379	36
10	Data ke 10	17:16:52.395	16
11	Data ke 11	17:16:52.421	26
12	Data ke 12	17:16:52.441	20
13	Data ke 13	17:16:52.486	45
14	Data ke 14	17:16:52.505	19
15	Data ke 15	17:16:52.523	18
16	Data ke 16	17:16:52.546	23
Rata-rata			22.67 ms

Berdasarkan hasil pengujian *latency* pada Tabel 4.2 dan Tabel 4.3, dapat dilihat bahwa waktu tunda pengiriman data menunjukkan nilai yang relatif kecil dan stabil. Pada percobaan pertama, nilai *latency* berkisar antara 10 ms hingga 35 ms dengan rata-rata sebesar 18,27 ms. Sementara itu, pada percobaan kedua diperoleh nilai *latency* antara 14 ms hingga 45 ms dengan rata-rata sebesar 22,67 ms.

Perbedaan rata-rata *latency* antara kedua percobaan menunjukkan adanya variasi kecil yang dapat disebabkan oleh proses komunikasi data, kondisi jaringan, maupun beban pemrosesan pada perangkat saat pengiriman data berlangsung. Meskipun demikian, seluruh data berhasil dikirim dengan jeda waktu yang relatif singkat sehingga sistem masih dapat dikategorikan memiliki performa komunikasi yang baik untuk kebutuhan pemantauan secara *real-time*. Nilai rata-rata *latency* yang berada di bawah 25 ms menunjukkan bahwa proses transmisi data berlangsung dengan respons yang cepat dan stabil.

Tabel 4. 4. Data *Latency* percobaan ke-3

No	Data ke-n	Waktu Terkirim (h:m:s.ms)	<i>Latency</i> (ms) $\Delta t = t_n - t_{n-1}$
1	Data ke 1	17:17:50.042	-
2	Data ke 2	17:17:50.070	24
3	Data ke 3	17:17:50.082	19
4	Data ke 4	17:17:50.092	15
5	Data ke 5	17:17:50.123	33
6	Data ke 6	17:17:50.135	17
7	Data ke 7	17:17:50.153	14
8	Data ke 8	17:17:50.170	15
9	Data ke 9	17:17:50.201	36
10	Data ke 10	17:17:50.218	16
11	Data ke 11	17:17:50.232	26
12	Data ke 12	17:17:50.248	20
13	Data ke 13	17:17:50.289	45
14	Data ke 14	17:17:50.307	19
15	Data ke 15	17:17:50.320	18
16	Data ke 16	17:17:50.330	23
Rata-rata			22.67 ms

Tabel 4. 5. Data *Latency* percobaan ke-4

No	Data ke-n	Waktu Terkirim (h:m:s.ms)	<i>Latency</i> (ms) $\Delta t = t_n - t_{n-1}$
1	Data ke 1	17:18:32.022	-
2	Data ke 2	17:18:32.045	23
3	Data ke 3	17:18:32.063	18
4	Data ke 4	17:18:32.079	16
5	Data ke 5	17:18:32.115	36
6	Data ke 6	17:18:32.134	19
7	Data ke 7	17:18:32.149	15
8	Data ke 8	17:18:32.161	12
9	Data ke 9	17:18:32.199	38
10	Data ke 10	17:18:32.215	16
11	Data ke 11	17:18:32.233	18
12	Data ke 12	17:18:32.250	17
13	Data ke 13	17:18:32.290	40
14	Data ke 14	17:18:32.308	18
15	Data ke 15	17:18:32.327	19
16	Data ke 16	17:18:32.347	20
Rata-rata			20.33 ms

Hasil pengujian pada Tabel 4.4 dan Tabel 4.5 menunjukkan bahwa sistem tetap mampu mempertahankan kinerja komunikasi yang baik. Pada percobaan ketiga diperoleh rata-rata *latency* sebesar 22,67 ms dengan rentang nilai antara 14 ms hingga 45 ms. Sedangkan pada percobaan keempat rata-rata *latency* menurun menjadi 20,33 ms dengan rentang nilai antara 12 ms hingga 40 ms.

Tabel 4. 6. Data *Latency* percobaan ke-5

No	Data ke-n	Waktu Terkirim (h:m:s.ms)	<i>Latency</i> (ms) $\Delta t = t_n - t_{n-1}$
1	Data ke 1	17:19:57.622	-
2	Data ke 2	17:19:57.635	23
3	Data ke 3	17:19:57.660	18
4	Data ke 4	17:19:57.672	16
5	Data ke 5	17:19:57.702	36
6	Data ke 6	17:19:57.720	19
7	Data ke 7	17:19:57.732	15
8	Data ke 8	17:19:57.747	12
9	Data ke 9	17:19:57.776	38
10	Data ke 10	17:19:57.793	16
11	Data ke 11	17:19:57.811	18
12	Data ke 12	17:19:57.826	17
13	Data ke 13	17:19:57.855	40
14	Data ke 14	17:19:57.870	18
15	Data ke 15	17:19:57.883	19
16	Data ke 16	17:19:57.899	20
Rata-rata			20.33 ms

Dari ke 5 percobaan diatas diperoleh rekapitulasi yang dapat dilihat sebagai berikut:

Tabel 4. 7. Rekapitulasi *latency* semua percobaan

Percobaan	Rata-rata (ms)	Min (ms)	Max (ms)
Percobaan 1	18.27	10	35
Percobaan 2	22.67	14	45
Percobaan 3	22.67	12	45
Percobaan 4	20.33	12	40
Percobaan 5	20.33	10	40
Rata rata keseluruhan	20.85		

Berdasarkan data kelima percobaan, diperoleh rata-rata *latency* keseluruhan sebesar 20,85 ms dengan nilai minimum 10 ms (Percobaan 1, Data ke-10) dan

nilai maksimum 45 ms (Percobaan 2 dan 3, Data ke-13). Rentang *latency steady-state* antar paket berada pada 10–45 ms, jauh lebih rendah dibandingkan pola percobaan terdahulu yang masih mengalami lonjakan besar di paket ke-2 akibat TCP *warm-up*. Hal ini mengindikasikan bahwa pada pengujian ini koneksi *socket* TCP/IP telah benar-benar stabil dari paket pertama, kemungkinan karena sesi *socket* dibuka lebih awal sebelum pengukuran dimulai. Temuan paling signifikan dari data kelima percobaan adalah adanya pola lonjakan *latency* yang berulang secara periodik pada paket-paket tertentu, sebagaimana ditunjukkan pada tabel berikut:

Tabel 4. 8. Pola periodik *latency*

Percobaan	Paket Lonjakan	Nilai <i>Latency</i> (ms)
Percobaan 1	Data ke-2, ke-5, ke-9, ke-13	35, 32, 32, 29
Percobaan 2	Data ke-5, ke-9, ke-13	33, 36, 45
Percobaan 3	Data ke-5, ke-9, ke-13	33, 36, 45
Percobaan 4	Data ke-5, ke-9, ke-13	36, 38, 40
Percobaan 5	Data ke-5, ke-9, ke-13	36, 38, 40

Hasil pengujian menunjukkan bahwa *latency* antar paket (Δt) tidak bersifat acak, melainkan membentuk pola periodik dengan lonjakan yang muncul secara konsisten pada interval tertentu, yaitu sekitar setiap empat paket. Pola ini mengindikasikan adanya pengaruh mekanisme internal pada sistem komunikasi, khususnya pada protokol TCP.

Berdasarkan penelitian (Al-Hammouri & Olsen, 2021) salah satu faktor utama yang menyebabkan fenomena tersebut adalah *Nagle's Algorithm*, yang berfungsi mengoptimalkan efisiensi jaringan dengan menggabungkan paket-paket kecil sebelum dikirim. Mekanisme ini menyebabkan penundaan sementara hingga kondisi tertentu terpenuhi, sehingga memunculkan lonjakan *latency* secara periodik. Selain itu, proses buffering pada sisi pengirim juga berkontribusi terhadap *delay*, di mana data disimpan sementara sebelum dikirim secara bersamaan dalam bentuk burst.

Interaksi antara *Nagle's Algorithm* dan mekanisme *delayed acknowledgement* (ACK) semakin memperkuat terjadinya penundaan ini. Kombinasi kedua mekanisme tersebut menyebabkan akumulasi *delay* yang

kemudian dilepaskan secara periodik, sehingga menghasilkan pola *latency* yang konsisten.

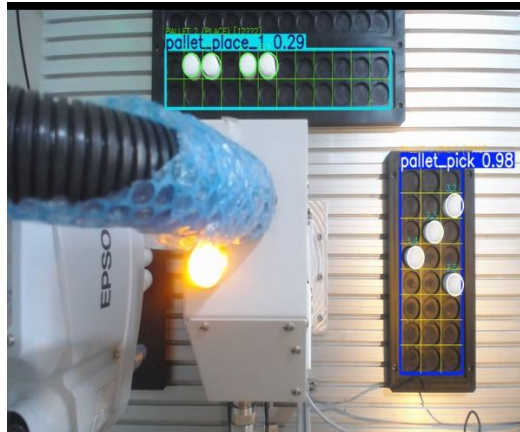
Dengan demikian, pola *delay* yang diamati merupakan konsekuensi dari strategi optimasi protokol TCP. Untuk aplikasi yang membutuhkan *latency* rendah dan respons *real-time*, mekanisme ini dapat diminimalkan dengan menonaktifkan *Nagle's Algorithm*, sehingga data dapat dikirim tanpa penundaan tambahan.

C. Pengujian dan analisis pengaruh *Cycle time*

Analisis *cycle time* pada penelitian ini tidak hanya bertujuan untuk mengukur waktu siklus kerja robot, tetapi juga untuk membandingkan performa antara sistem berbasis kamera (*vision system*) dan sistem konvensional tanpa kamera pada kondisi kerja yang sama. Perbandingan ini dilakukan untuk mengevaluasi sejauh mana penggunaan kamera dalam proses deteksi objek mempengaruhi efisiensi waktu keseluruhan sistem, khususnya pada aplikasi *pick and place*.

Pada sistem konvensional, posisi *workpiece* telah ditentukan secara tetap (*fixed position*), sehingga robot dapat langsung melakukan proses *pick and place* tanpa proses identifikasi tambahan. Sebaliknya, pada sistem berbasis kamera, robot harus melalui tahapan tambahan berupa akuisisi citra, pemrosesan data, serta pengiriman koordinat hasil deteksi sebelum melakukan pergerakan. Tahapan ini berpotensi menambah waktu siklus, namun memberikan fleksibilitas yang lebih tinggi terhadap variasi posisi objek.

Pengujian dilakukan sebanyak lima kali percobaan untuk masing-masing metode dengan kondisi yang identik, yaitu mendeteksi empat buah *workpiece* dalam satu siklus yang dapat dilihat pada gambar 4.6. Dengan demikian, dalam setiap percobaan robot melakukan empat kali proses *pick* dan empat kali proses *place*, kemudian kembali ke posisi home sebagai akhir siklus. Variasi parameter dilakukan pada kecepatan robot yang diubah dari 50 hingga 100, sementara nilai percepatan (*acceleration* dan *deceleration*) dijaga konstan pada nilai 50. Dengan metode ini, analisis difokuskan pada pengaruh perubahan kecepatan terhadap *cycle time* tanpa dipengaruhi oleh variabel percepatan.



Gambar 4. 6. Kondisi *workpiece* saat pengujian *cycle time*

Setelah pengujian diperoleh data perbandingan antara penggunaan sistem vision robotik dengan tanpa sistem *vision* robotik sebagai berikut:

Tabel 4. 9. Hasil pengujian *cycle time* dengan *vision*

Speed (%)	Cycle time				
	Percobaan 1 (s)	Percobaan 2 (s)	Percobaan 3 (s)	Percobaan 4 (s)	Percobaan 5 (s)
50	29.61	29.2	29.32	30.01	29.52
60	26.8	26.6	26.5	26.6	26.6
70	25.6	25.4	25.5	25.5	25.4
80	24.7	25.0	24.8	24.9	24.8
90	23.6	23.5	23.9	23.5	23.4
100	22.5	22.5	22.8	22.6	22.6

Tabel 4. 10. Hasil pengujian *cycle time* tanpa *vision*

Speed (%)	Cycle time				
	Percobaan 1 (s)	Percobaan 2 (s)	Percobaan 3 (s)	Percobaan 4 (s)	Percobaan 5 (s)
50	29.1	28.8	28.9	29.4	29.0
60	26.3	26.1	26.1	26.0	26.2
70	25.1	24.9	25.0	25.1	24.9
80	24.2	24.5	24.3	24.4	24.3
90	23.1	23.0	23.4	23.0	22.9
100	22.0	22.1	22.2	22.1	22.1

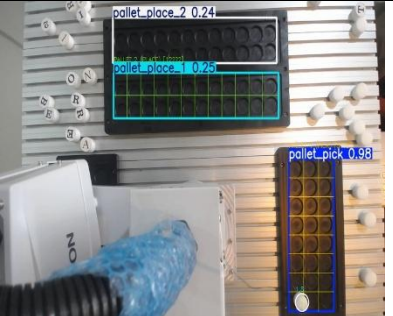
Hasil perbandingan menunjukkan bahwa perbedaan *cycle time* antara sistem berbasis *vision* dan sistem konvensional relatif kecil. Hal ini disebabkan karena proses deteksi objek dan pengiriman koordinat pada sistem *vision* dilakukan sebelum robot memulai pergerakan *pick and place*, dengan waktu eksekusi

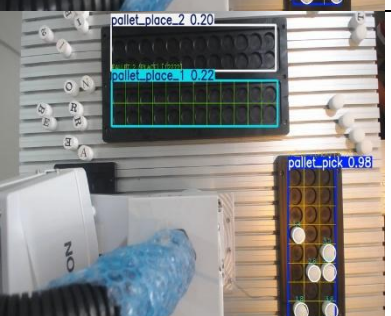
kurang dari 1 detik. Dengan demikian, tambahan waktu dari sistem *vision* tidak secara signifikan mempengaruhi total *cycle time*.

D. Pengujian dan hasil deteksi *pallet* dan *workpiece*

Pengujian dilakukan untuk mengevaluasi kemampuan sistem dalam mendeteksi jumlah *workpiece* pada dua kondisi, yaitu *pallet pick* dan *pallet place 1*. Setiap skenario diuji sebanyak 5 kali dengan variasi jumlah *workpiece* dari 1 hingga 10. Tujuan pengujian ini adalah untuk mengetahui tingkat akurasi sistem dalam menghitung jumlah objek serta konsistensi hasil deteksi pada kondisi yang berbeda. Hasil pengujian disajikan pada tabel berikut:

Tabel 4. 11. Hasil pengujian deteksi *pallet pick*

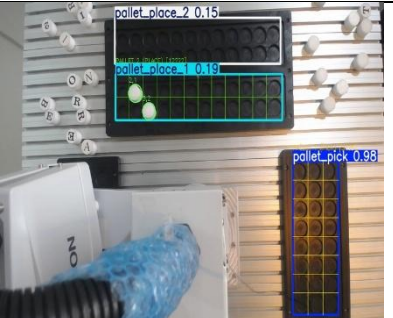
<i>Workpiece pallet pick</i> (aktual)	Jumlah <i>workpiece pallet pick</i> (deteksi)					Hasil
	Uji ke-1	Uji ke-2	Uji ke-3	Uji ke-4	Uji ke-5	
1	1	1	1	1	1	
2	2	2	2	2	2	
3	3	3	3	3	3	

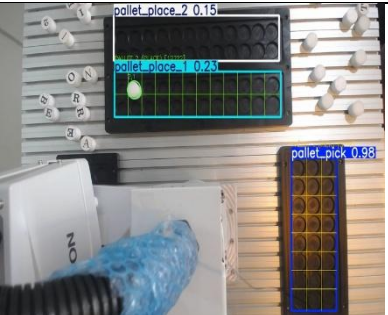
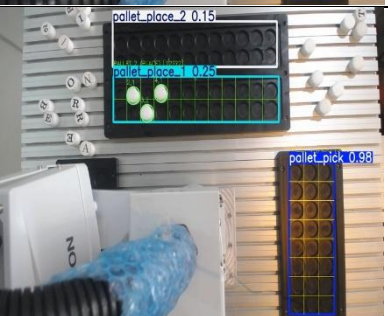
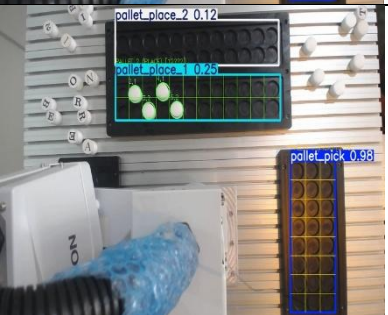
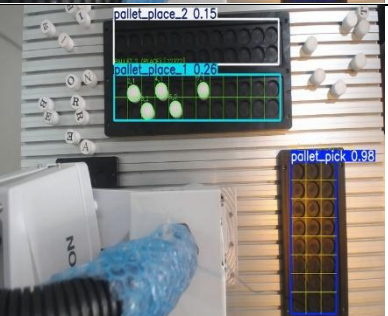
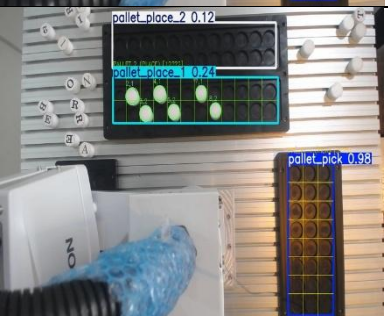
4	4	4	4	4	4	
5	5	5	5	5	5	
6	6	6	6	6	6	
7	7	7	7	7	7	
8	8	8	8	8	8	

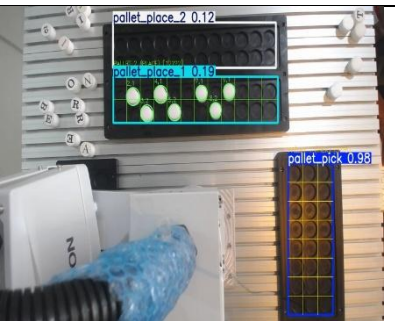
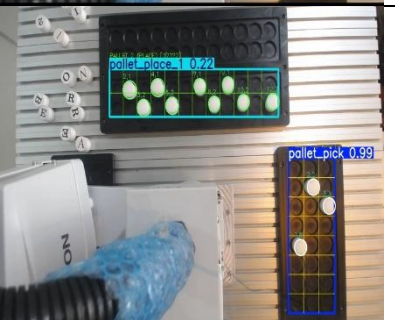
9	9	9	9	9	9	
10	10	10	10	10	10	

Berdasarkan pengujian yang telah dilakukan pada kondisi pallet pick sebanyak 5 kali untuk setiap variasi jumlah *workpiece* dari 1 hingga 10, diperoleh hasil bahwa sistem mampu mendeteksi *pallet* dan *workpiece* dengan sangat baik pada seluruh pengujian. Seluruh objek yang berada pada area pallet pick berhasil teridentifikasi sesuai dengan jumlah aktual tanpa adanya kesalahan deteksi. Selanjutnya pada tabel berikut hasil pengujian deteksi *workpiece* pada *pallet place*.

Tabel 4. 12. Hasil pengujian deteksi *pallet place*

<i>Workpiece pallet place</i> (aktual)	Jumlah <i>workpiece pallet place</i> (deteksi)					Hasil
	Uji ke-1	Uji ke-2	Uji ke-3	Uji ke-4	Uji ke-5	
1	1	1	1	1	1	

2	2	2	2	2	2	
3	3	3	3	3	3	
4	4	4	4	4	4	
5	5	5	5	5	5	
6	6	6	6	6	6	

7	7	7	7	7	7	
8	8	8	8	8	8	
9	9	9	9	9	9	
10	10	10	10	10	10	

Berdasarkan pengujian yang telah dilakukan pada kondisi *pallet place* sebanyak 5 kali percobaan untuk setiap variasi jumlah *workpiece* dari 1 hingga 10, diperoleh hasil bahwa sistem mampu mendeteksi *pallet* dan *workpiece* dengan sangat baik pada seluruh skenario pengujian. Seluruh objek yang berada pada area *pallet place* berhasil teridentifikasi sesuai dengan jumlah aktual tanpa adanya kesalahan deteksi maupun *missed detection* pada seluruh 50 percobaan yang dilakukan.

Hasil ini menunjukkan bahwa model YOLO26 yang telah dilatih mampu mengenali dan membedakan *area pallet place* secara konsisten meskipun terdapat variasi jumlah *workpiece* yang ditempatkan. Tidak ditemukan kasus *false positive* maupun *false negative* pada seluruh pengujian kondisi *pallet place*, yang mengindikasikan bahwa tingkat akurasi deteksi pada kondisi ini mencapai 100%. Konsistensi hasil yang sempurna pada seluruh 5 kali pengulangan untuk setiap variasi jumlah objek membuktikan bahwa sistem memiliki keandalan yang tinggi dalam mendeteksi *workpiece* pada *area pallet place* dalam kondisi pencahayaan dan posisi kamera yang telah ditetapkan.

BAB V

KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem *vision-guided robotics* (VGR) berbasis YOLO26 untuk kontrol robot SCARA T3 pada proses *pick and place* dapat ditarik simpulan sebagai berikut:

1. Sistem *vision-guided robotics* (VGR) berbasis YOLO26 berhasil dirancang dan diimplementasikan pada robot SCARA T3 untuk mendukung proses *pick and place* secara otomatis. Model YOLO26 yang telah dilatih mampu mendeteksi dan mengklasifikasikan objek *pallet* dengan tingkat akurasi yang sangat baik, sehingga dapat digunakan sebagai sumber informasi posisi objek bagi robot dalam menjalankan proses pengambilan dan penempatan benda kerja.
2. Algoritma transformasi koordinat dan mekanisme komunikasi yang digunakan berhasil menghubungkan sistem *vision* dengan robot SCARA T3. Hasil deteksi objek dari kamera dapat dikonversi menjadi koordinat *grid* yang sesuai dengan area kerja robot, sehingga robot dapat bergerak menuju posisi target secara tepat. Hasil pengujian menunjukkan bahwa sistem mampu mengidentifikasi posisi *workpiece* pada *pallet pick* maupun *pallet place* secara konsisten.
3. Sistem yang dikembangkan mampu mendukung operasi *pick and place* secara *real time* dengan performa komunikasi yang baik. Rata-rata *latency* komunikasi yang rendah menunjukkan bahwa pertukaran data antara komputer dan robot berlangsung responsif. Selain itu, penerapan sistem *vision* hanya memberikan tambahan waktu yang sangat kecil terhadap *cycle time* proses, sehingga efisiensi operasional tetap terjaga. Dengan kemampuan deteksi yang akurat, komunikasi yang andal, dan waktu respons yang cepat.

B. Saran

Berdasarkan hasil penelitian dan pengujian yang telah dilakukan, terdapat beberapa saran untuk pengembangan lebih lanjut:

1. Peningkatan Akurasi Deteksi pada Variasi Pencahayaan

Sistem deteksi lingkaran berbasis HSV yang digunakan dalam penelitian ini masih sensitif terhadap perubahan kondisi pencahayaan. Untuk meningkatkan robustitas sistem, disarankan penggunaan pencahayaan LED terkontrol yang dipasang secara permanen pada area kerja, atau menggantikan segmentasi HSV dengan metode deteksi berbasis *deep learning* yang lebih adaptif terhadap variasi pencahayaan lingkungan industri yang dinamis.

2. Penonaktifan *Nagle's Algorithm* untuk Optimasi *Latency*

Hasil pengujian menunjukkan adanya pola lonjakan *latency* periodik setiap empat paket akibat mekanisme *Nagle's Algorithm* dan *delayed acknowledgement* pada protokol TCP. Untuk aplikasi yang membutuhkan respons *real-time* lebih ketat, disarankan menonaktifkan *Nagle's Algorithm* melalui pengaturan opsi `TCP_NODELAY` pada konfigurasi socket Python, sehingga setiap paket data koordinat dapat dikirimkan segera tanpa penundaan akumulasi *buffer*.

3. Perluasan Kelas Objek dan Variasi Bentuk *Workpiece*

Penelitian ini hanya menguji deteksi *workpiece* berbentuk lingkaran pada tiga kelas *pallet* yang spesifik. Untuk aplikasi industri yang lebih luas, disarankan pengembangan dataset dengan variasi bentuk *workpiece* yang lebih beragam (persegi, segitiga, tidak beraturan) serta penambahan kelas *pallet* baru, sehingga sistem dapat menangani kondisi *flexible manufacturing* yang sesungguhnya.

DAFTAR PUSTAKA

- Aini, M., Aini, M., Fitrianingrum, A. M., Pamungkas, Z. S., Rungkat, J. A., & Kurniahtunnisa. (2025). Engineering Design Process (EDP) Learning Model on Learning Outcomes, Critical Thinking and Communication Skills of Science Education Students. *Jurnal Iqra'*, 10(2).
- Al-Hammouri, A. T., & Olsen, R. L. (2021). A dataset for buffering delays due to the interaction between the Nagle algorithm and the delayed acknowledgement algorithm in cyber-physical systems communication. *Data in Brief*, 39, 107530.
- Bochkovski, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. *Electronics*, 11(15), 2348. <http://arxiv.org/abs/2004.10934>
- Craig, J. J. (2005). *Introduction to Robotics: Mechanics and Control* (3rd ed.). Pearson Prentice Hall.
- Diwan, T., Anirudh, G., & Tembhurne, J. V. (2022). Object detection using YOLO: challenges, architectural successors, datasets and applications. *Multimedia Tools and Applications*, (82), 9243–9275.
- Djarmika, G. H. (2023). Peran Transformasi Digital Dalam Meningkatkan Efisiensi Operasional Pada Industri Manufaktur Di Indonesia. *Tartib: Journal of Educational Management*, 4(1), 56–81.
- Firdaus, A. Z. (2025). Sistem Klasifikasi Sampah Otomatis Berbasis Deteksi Objek Real-Time Pada Single Board Computer Dengan Algoritma YOLO. *IJEIS (Indonesian Journal of Electronics and Instrumentation Systems)*, 15(1).
- Gafari, A., Jurinaldo, D., & Ramadhanu, A. (2025). Klasifikasi Otomatis Sampah Kering Menggunakan HSV Color Space, Segmentasi Morfologi, dan Extreme Learning Machine. *JATI (Jurnal Mahasiswa Teknik Informatika)*, 9(5), 7674–7681.
- Gheorghe, C., Duguleana, M., Boboc, R. G., & Postelnicu, C. C. (2024). Analyzing Real-Time Object Detection with YOLO Algorithm in Automotive Applications: A Review. *CMES - Computer Modeling in Engineering and Sciences*, 141(3), 1939–1981. <https://doi.org/10.32604/cmes.2024.054735>

- Guan, Z. (2023). Real time object recognition based on YOLO model. *Theoretical and Natural Science*, 28(1), 137–143. <https://doi.org/10.54254/2753-8818/28/20230450>
- Hikmahwan, B., Fu'adi, A., Putra, B. J. M., & Nugroho, B. Y. (2024). Mendeteksi Objek Bulat Secara Real-Time Menggunakan Model Warna HSV Berbasis Android. *Journal of Electrical, Electronic, Mechanical, Informatic and Social Applied Science*, 3(1), 9–15. <https://doi.org/10.58991/eemisas.v3i1.51>
- Izzulhaq, A. I., & Budiyanto, M. (2025). Rancang Bangun Sistem Otomasi Paleletizinf Menggunakan Model Pemrograman Industri Berorientasi Obyek. *Jurnal Teknologi Dan Vokasi*, 3(2), 185–194.
- Kang, S., Hu, Z., Liu, L., Zhang, K., & Cao, Z. (2025). Object Detection YOLO Algorithms and Their Industrial Applications: Overview and Comparative Analysis. *Electronics (Switzerland)*, 14(6), 1–36. <https://doi.org/10.3390/electronics14061104>
- Kapusi, T. P., Erdei, T. I., Husi, G., & Hajdu, A. (2022). Application of Deep Learning in the Deployment of an Industrial SCARA Machine for Real-Time Object Detection. *Robotics*, 11(4). <https://doi.org/10.3390/robotics11040069>
- Kim, S., & Lee, K. (2023). The paradigm shift of mass customisation research. *International Journal of Production Research*, 61(10), 3350–3376. <https://doi.org/10.1080/00207543.2022.2081629>
- Kivrak, O., & Gurbuz, M. Z. (2022). Perfomance Comparison of YOLOv3, YOLOv4, YOLOv5, Alghorithms: A Case Study for Poultry Recognition. *European Journal of Science and Technology*, (38), 392–397.
- Ku, B., Kim, K., & Jeong, J. (2022). Real-Time ISR-YOLOv4 Based Small Object Detection for Safe Shop Floor in Smart Factories. *Electronics (Switzerland)*, 11(15), 1–18. <https://doi.org/10.3390/electronics11152348>
- Olorunshola, O., Jemitola, P., & Ademuwagun, A. (2023). Comparative Study of Some Deep Learning Object Detection Algorithms: R-CNN, FAST R-CNN, FASTER R-CNN, SSD, and YOLO. *Nile Journal of Engineering and Applied Science*, 1(0), 1. <https://doi.org/10.5455/njeas.150264>
- Permana, Y. P., Kom, S., & Kom, M. (2025). *Kecerdasan Buatan: Mengubah*

Dunia dengan Teknologi. MEGA PRESS NUSANTARA.

- Pramudita, R., Ramadhan, M. A. P., Ashari, M. R., Nafisa, R. A., & Rahmawati, D. N. (2024). Analisis Dampak Otomasi Industri terhadap Efisiensi Operasional dan Optimasi Konsumsi Energi. *Jurnal Ilmiah Teknologi Infomasi Terapan*, 11(1).
- Rahmadhany, J. X., & Prasetio, B. H. (2025). Penerapan Circular Hough Transform Untuk Deteksi Sel Darah Pada Citra Mikroskopik Berbasis Smartphone. *Jurnal Pengembangan Teknologi Informasi Dan Ilmu Komputer*, 9(3).
- Robinson, N., Tidd, B., Campbell, D., Kulic, D., & Corke, P. (2023). Robotic vision for human-robot interaction and collaboration: A survey and systematic review. *ACM Transactions on Human-Robot Interaction*, 12(1), 1–66. <https://doi.org/10.1145/3570931>
- Roy, S., Mondal, R., Paoletti, M. E., Haut, J. M., & Plaza, A. (2021). Morphological Convolutional Neural Networks for Hyperspectral Image Classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 14, 8689–8702. <https://doi.org/10.1109/JSTARS.2021.3088228>
- Sapkota, R., Flores-Calero, M., Qureshi, R., Badgujar, C., Nepal, U., Poullose, A., Zeno, P., Vaddevolu, U. B. P., Khan, S., Shoman, M., Yan, H., & Karkee, M. (2025). YOLO advances to its genesis: a decadal and comprehensive review of the You Only Look Once (YOLO) series. *Artificial Intelligence Review*, 58(9). <https://doi.org/10.1007/s10462-025-11253-3>
- Sapkota, R., & Karkee, M. (2025). *Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 Object Detectors for Computer Vision and Pattern Recognition*. 26, 1–16. <http://arxiv.org/abs/2510.09653>
- Sari, N. P. (2024). Analisis Performa Algoritma CNN dalam Klasifikasi Citra Medis Berbasis Deep Learning. *Jurnal Komputer*, 2(2), 87–92.
- Sarma, K. S. R. K., Sasikala, C., Surendra, K., Erukala, S., & Aruna, S. L. (2024). A comparative study on faster R-CNN, YOLO and SSD object detection algorithms on HIDS system. *AIP Conference Proceedings*, 2971(1), 60044. <https://doi.org/10.1063/5.0195857>
- Shende, R., & Khandelwal, S. (2025). YOLO Vs Faster RCNN for Object Detection

- and Recognition. *International Research Journal of Multidisciplinary Scope*, 6(3), 1611–1621. <https://doi.org/10.47857/irjms.2025.v06i03.04661>
- Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, Planning and Control*. Springer-Verlag London. <https://doi.org/10.1007/978-1-84628-642-1>
- Singh, A., Kalaichelvi, V., & Karthikeyan, R. (2022). A survey on vision guided robotic systems with intelligent control strategies for autonomous tasks. *Cogent Engineering*, 9(1), 2050020. <https://doi.org/10.1080/23311916.2022.2050020>
- Sutarti, Siswanto, & Putri Jutika, A. (2022). Implementasi Face Recognition Berbasis Haar-Cascade Classifier Pada Sistem Keamanan Rumah Menggunakan Dual-Camera. *INFOTECH Journal*, 8(2), 106–115.
- Wu, W., Liu, H., Li, L., Long, Y., Wang, X., Wang, Z., Li, J., & Chang, Y. (2021). Application of local fully Convolutional Neural Network combined with YOLO v5 algorithm in small target detection of remote sensing image. *PLoS ONE*, 16(10 October), 1–15. <https://doi.org/10.1371/journal.pone.0259283>
- Ye, Z., & Yu, J. (2021). Deep morphological convolutional network for feature learning of vibration signals and its applications to gearbox fault diagnosis. *Mechanical Systems and Signal Processing*, 161, 107984. <https://doi.org/10.1016/j.ymssp.2021.107984>
- Zhao, K., Wang, Y., Zuo, Y., & Zhang, C. (2022). Palletizing Robot Positioning Bolt Detection Based on Improved YOLO-V3. *Journal of Intelligent & Robotic Systems*, 104(41).
- Zhong, H., & Wang, R. (2022). A visual-degradation-inspired model with HSV color-encoding for contour detection. *Journal of Neuroscience Methods*, 369, 109423. <https://doi.org/10.1016/j.jneumeth.2021.109423>

LAMPIRAN

Lampiran 1. Program Epson RC+



Lampiran 2. Repository Github



Lampiran 3. Buku Panduan

